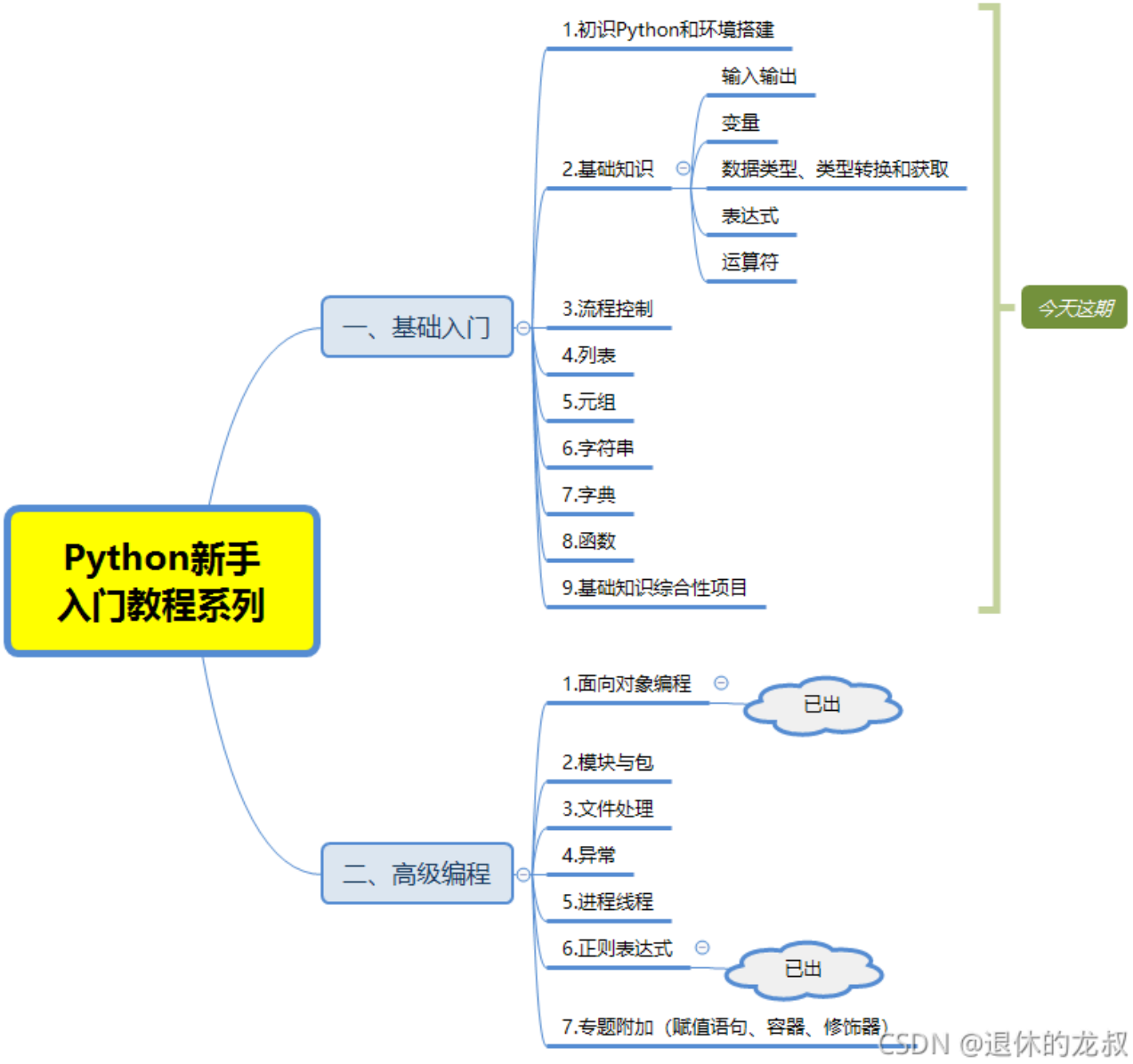
万字【Python基础】保姆式教学，零基础快速入门Python_西安科技大学研究生院-CSDN博客

本文带你快速入门Python，从语言介绍、安装指南、基本语法、数据类型、运算符到流程控制、列表、元组、字符串、字典、函数等知识点，适合初学者循序渐进学习。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

前言

又是爆肝干文的日子，继上次说要出一期Python新手入门教程系列文章后，就在不停地整理和码字，终于是把【基础入门】这一块给写出来了。



高级编程的【正则表达式】和【面向对象编程】内容我在之前已经出过了，感兴趣可以去看看我的专栏，今天的内容是【基础入门】。

【】

不积跬步无以至千里，不积小流无以成江海，一个一个板块的知识积累，早晚你也会成为你羡慕的那种人，接下来就是正式的内容了。

今日内容预览

一、初识Python

1.什么是Python

Python是1门程序设计语言。在开发者眼里，语言可以分为3类：

1. 自然语言：人能听懂的语言，例如汉语，英语，法语等等。
2. 机器语言：机器能听懂的语言，机器只能听懂0和1。
3. 程序设计语言：机器能够听懂，人能听懂的语言，例如Python，C，C++ 、C# 、Java等等。

同样的，在开发者眼里还可以分为高级程序设计语言和低级程序设计语言。越接近于人类的语言越高级
，例如Python；越接近于 机器语言 越低级，例如汇编就属于低级程序员设计语言。

2.Python 的历史

喝水不忘挖井人，在正式学习Python之前我们很有必要了解一下Python的过去和大家的祖师爷。

Python是1989年由Guido van Rossum 在圣诞节期间创建的，国内程序员比较喜欢简单但的名字，所以

取他名字的前三个字母gui，人送外号龟叔。Python名字的由来也是源于龟叔的喜好，因为当时的龟叔比较喜欢《蒙提·派森的飞行马戏团》这个小品，所以就把他创建的这门语言叫做Python。

龟叔今年才65岁，头发还是很多的，所以学Python的小伙伴也不用太担心秃头。龟叔我记得去年时候还加入了微软的开发部，一把年纪了还在一线写代码，着实让人佩服。

3. Python的特点

Python的特点主要有语法简洁、类库强大、胶水语言（调用其他语言的类库）、代码量较少等特点，这个简单了解一下就可以了，后面用到了你就会明白的。

> （谁用谁知道）

4.Python 运行机制

程序运行主要有两种机制：编译型和解释型。编译型是将代码（源文件）通过编译器，编译成机器码文件，当运行的时候直接执行机器码文件，例如C语言；解释型是将源文件通过解释器，逐行进行翻译并运行。

Python则属于解释型的语言。

编译型和解释型语言各有优缺点：

- 解释型：

  缺点：执行慢
  优点：可以跨平台（操作系统）

- 编译型：

  缺点：不能跨平台
  优点：执行快

5.软件安装

新手学Python我建议装一下Python和Pycharm，在以后写代码的时候会经常用到这两款软件，这两款软件都可以在官网下载，如果你不想去官网下载，我这边也有下好的安装包，自取之后按照安装流程开始即可。

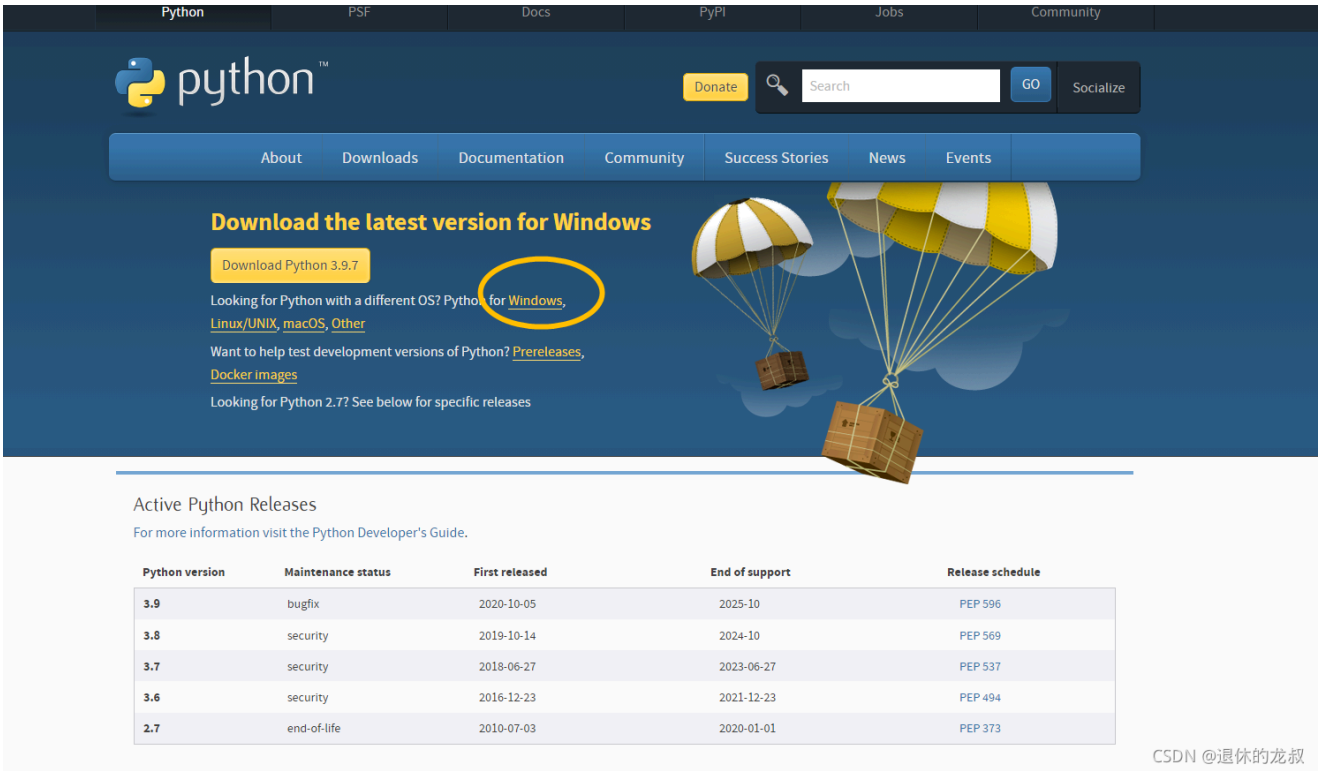
> 软件包下载链接：https://pan.baidu.com/s/1BtnMGsfYNFIkmJ26wuGjCw
> 提取码：al6e

（1）Python的安装步骤

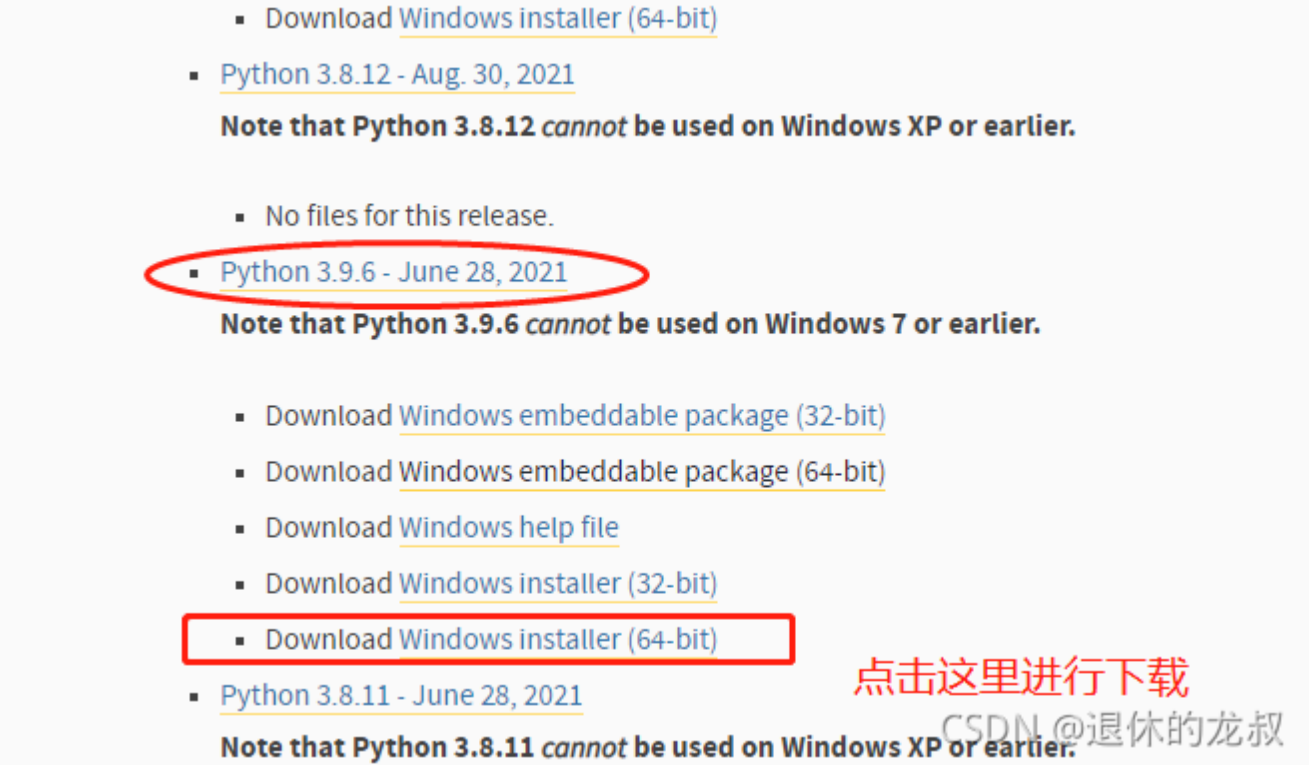
Python的安装可以去官网直接下载，由于是国外的网站，可能响应和下载速度都会有点慢，如果你不着急就等着就行，想快一点的话用迅雷来下载就很快。

- **下载Python**

下载地址是 ttps://www.python.org/downloads/ ，也可以从官网点Downloads 进入下载页面，选择你的电脑对应的版本，我的是Windows电脑，所以就直接点击 Windows 。

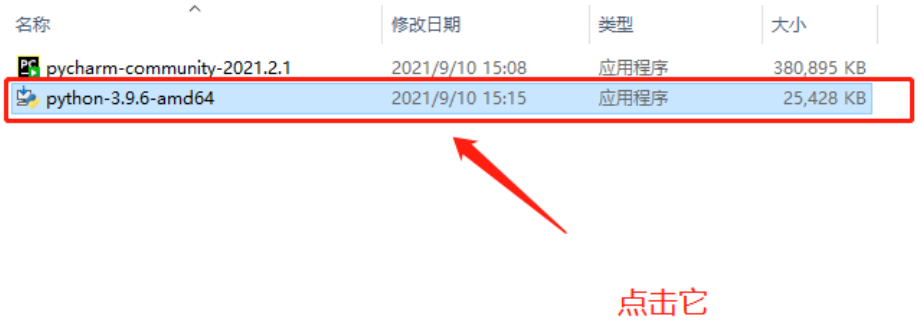


Python目前是出到了3.9.7版本，你可以看到有很多版本的选择可下载，我用的是3.9.5这个版本，然后根据电脑挑选型号，我的电脑是64位的，所以就下载64位的软件。



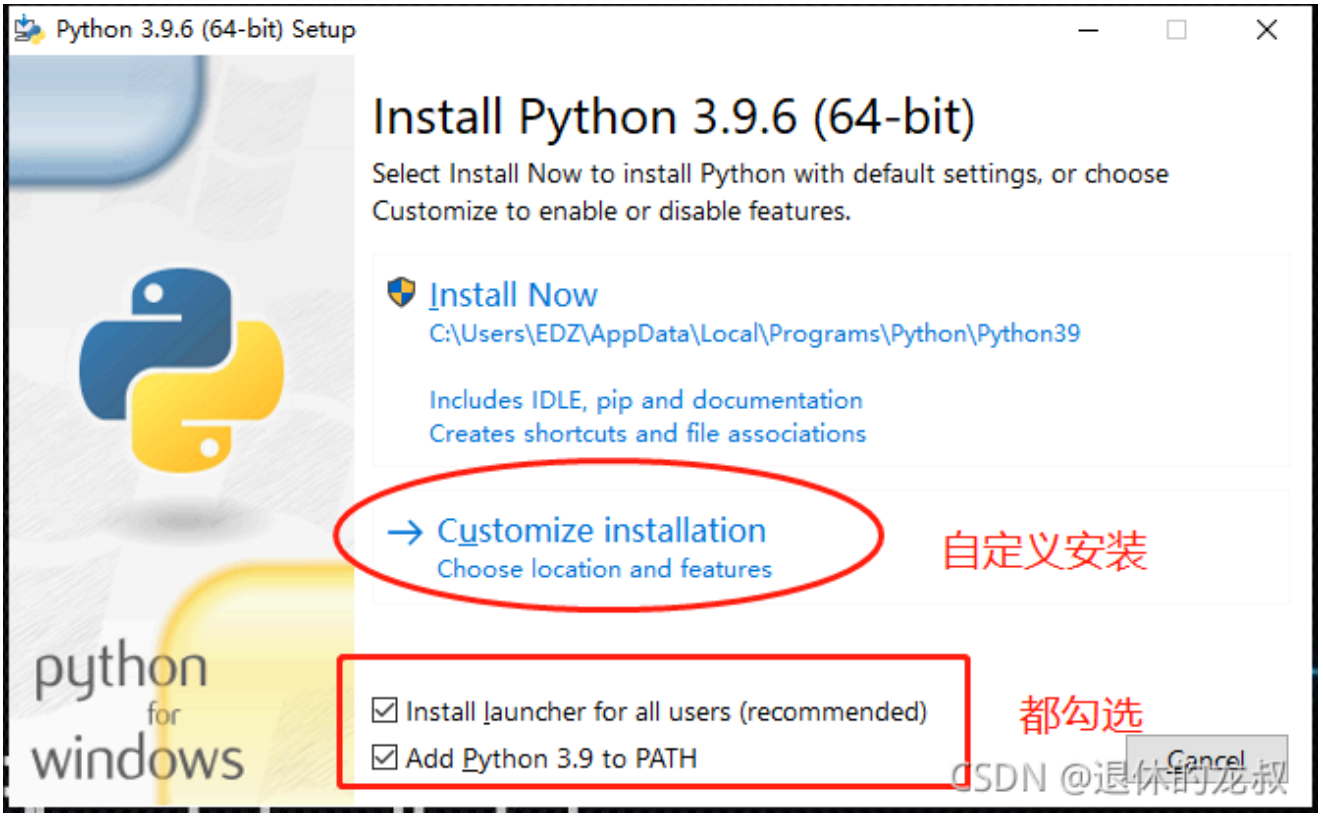
- 安装Python

双击下载好的文件，进入安装界面。

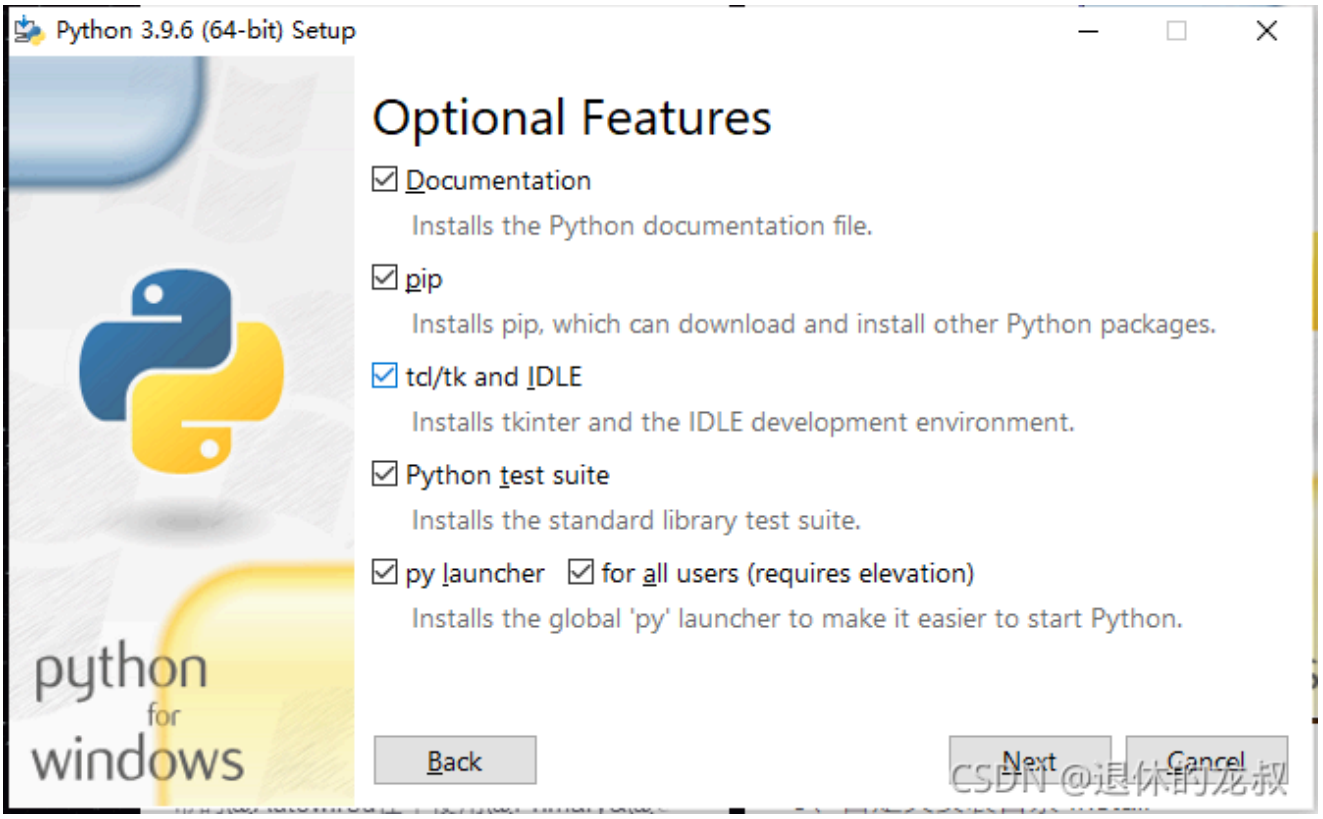


CSDN @退休的龙叔

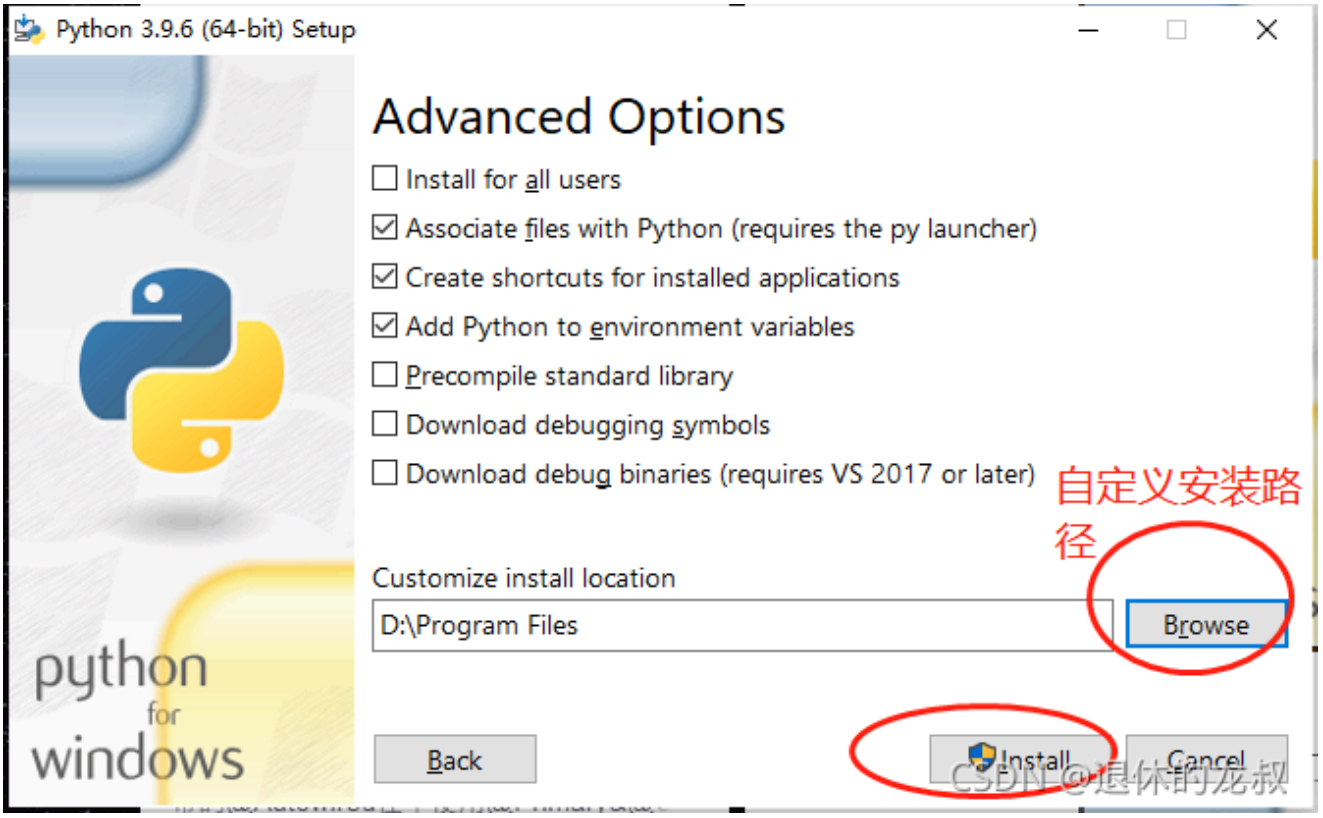
选择自定义安装，把两个选择都勾选就能将Python安装到PATH中。



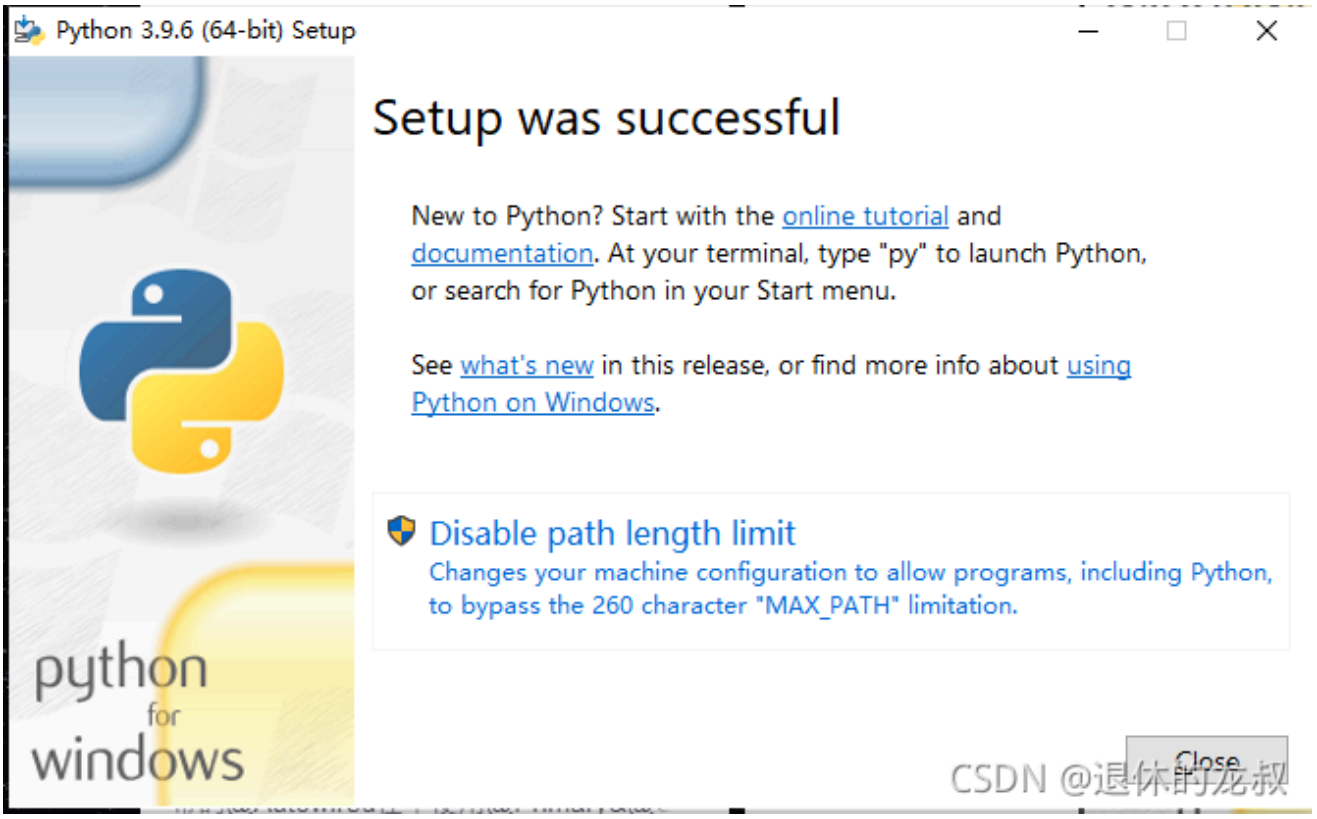
这个界面不变动，直接点 Next 。



选择一个你要安装的路径，不然它会把你装到C盘，我把它安装到了D盘，然后点击 Install 。

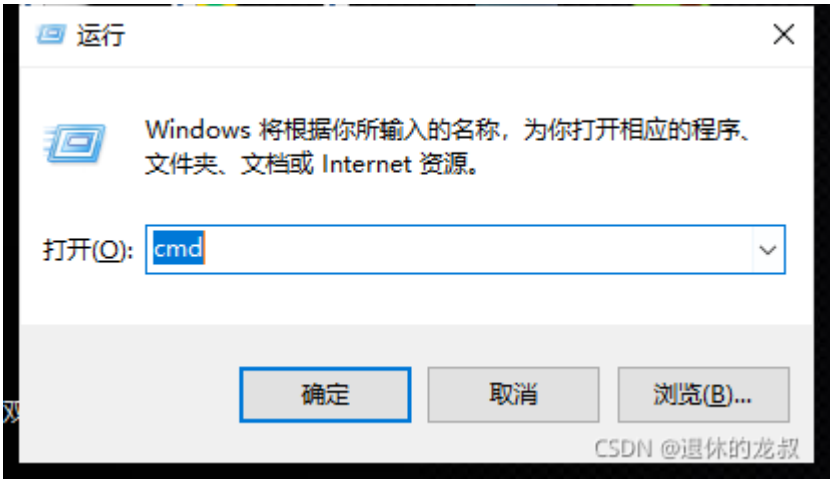


安装完成，点击 Close 。

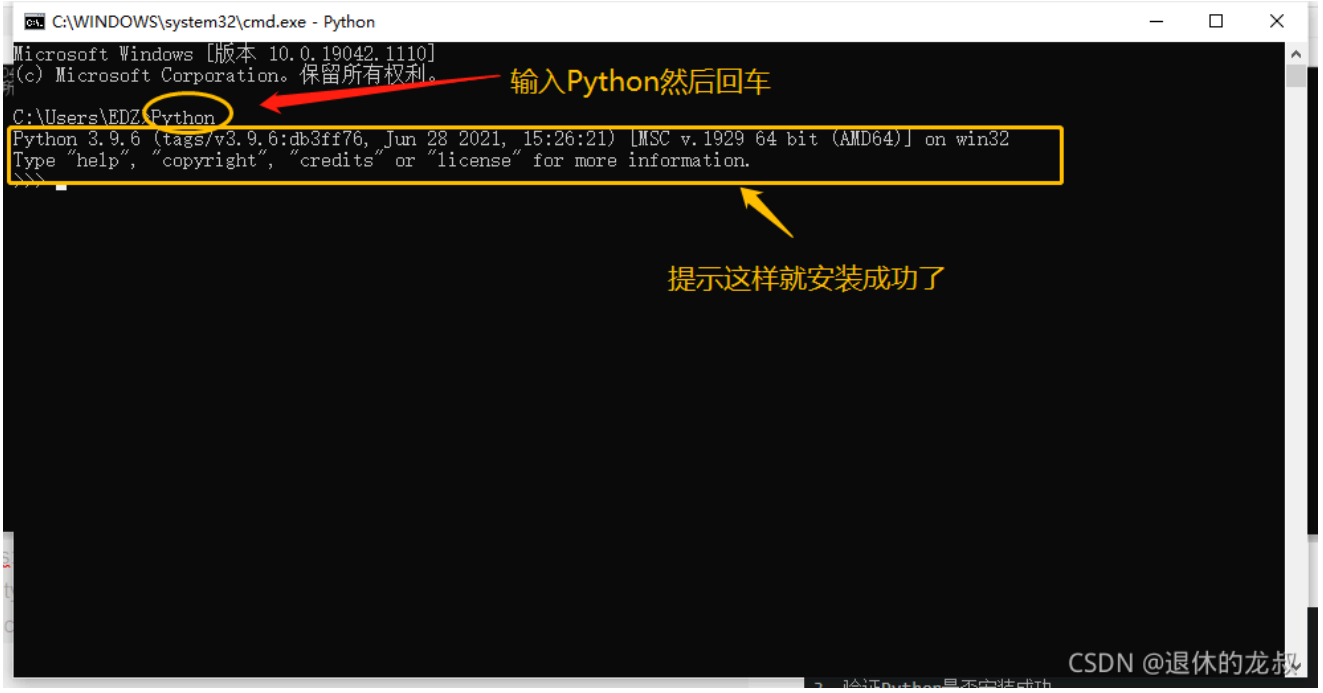


- **Python测试**

测试一下你的Python是否安装成功，WIN + R 键调出命令窗口，输入 cmd 。

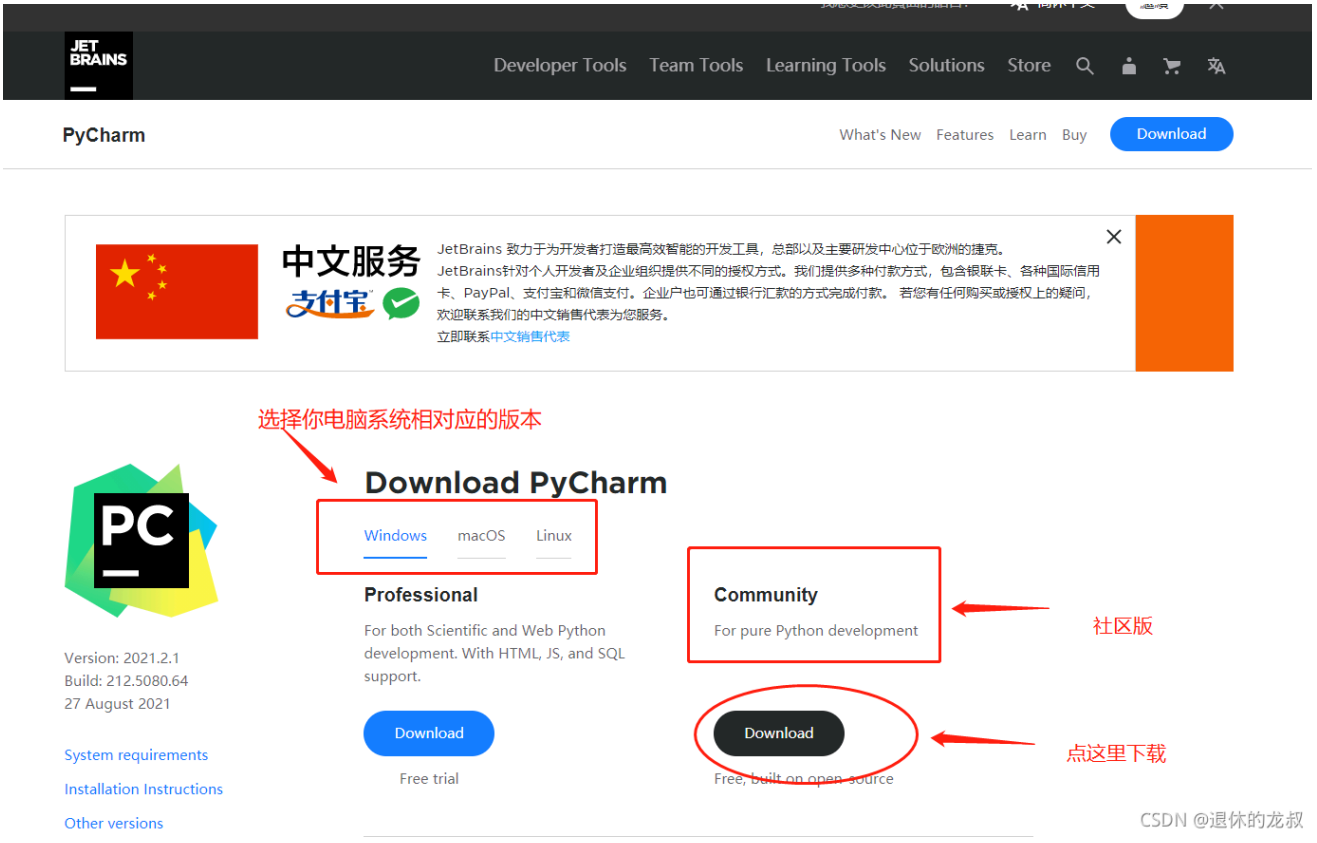


在黑窗口输入 Python,然后回车，如果提示Python 3.9.6 等信息就说明已经安装成功了，如果没有提示，那就回去看看哪个环境你出错了。

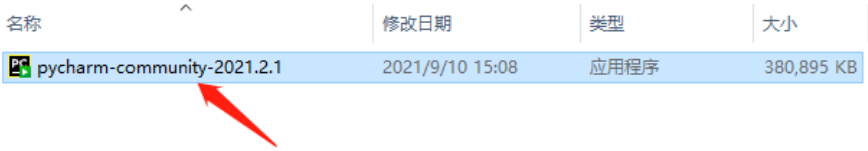


（2）Pycharm

Pycharm官网下载，选择一下系统，我是Windows系统所以默认不变，然后下载社区版的就可以了，等你变强大以后再用专业版的。

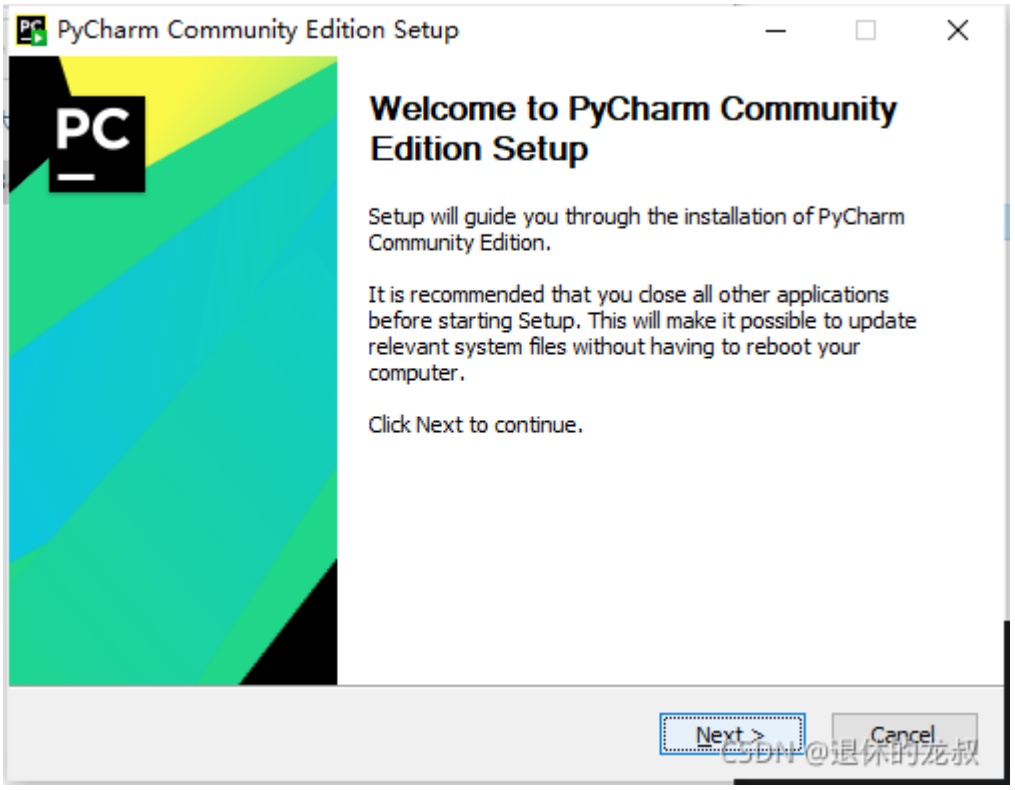


下载好之后，双击开始安装。

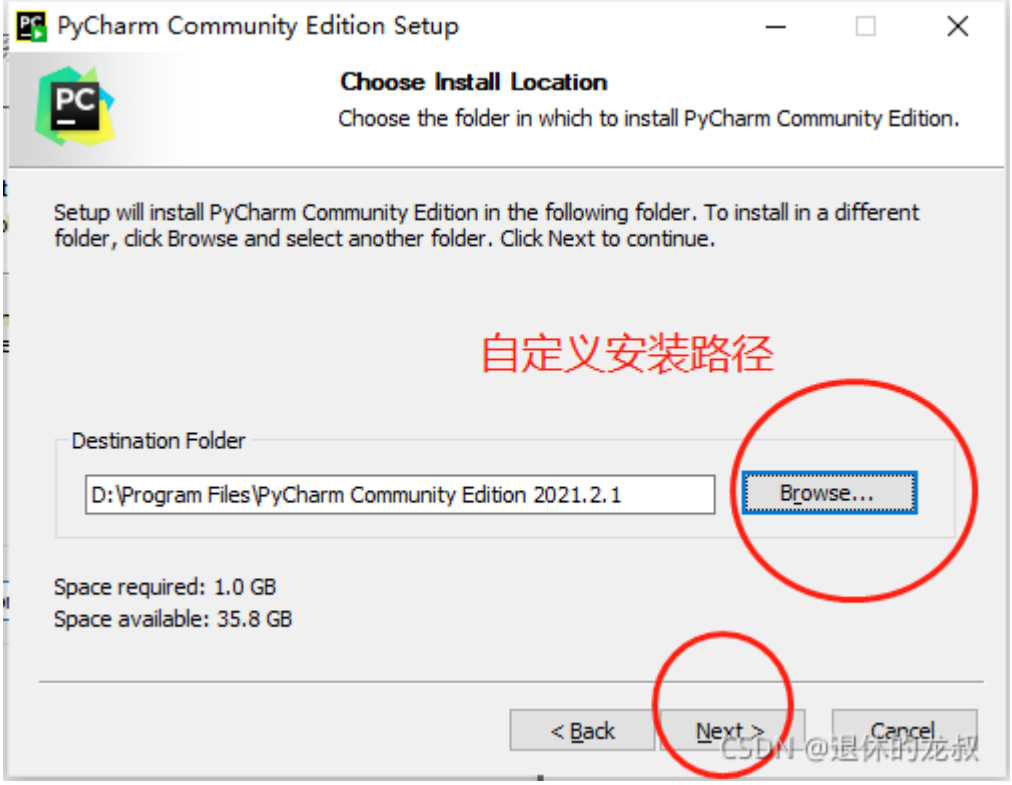


CSDN @退休的龙叔

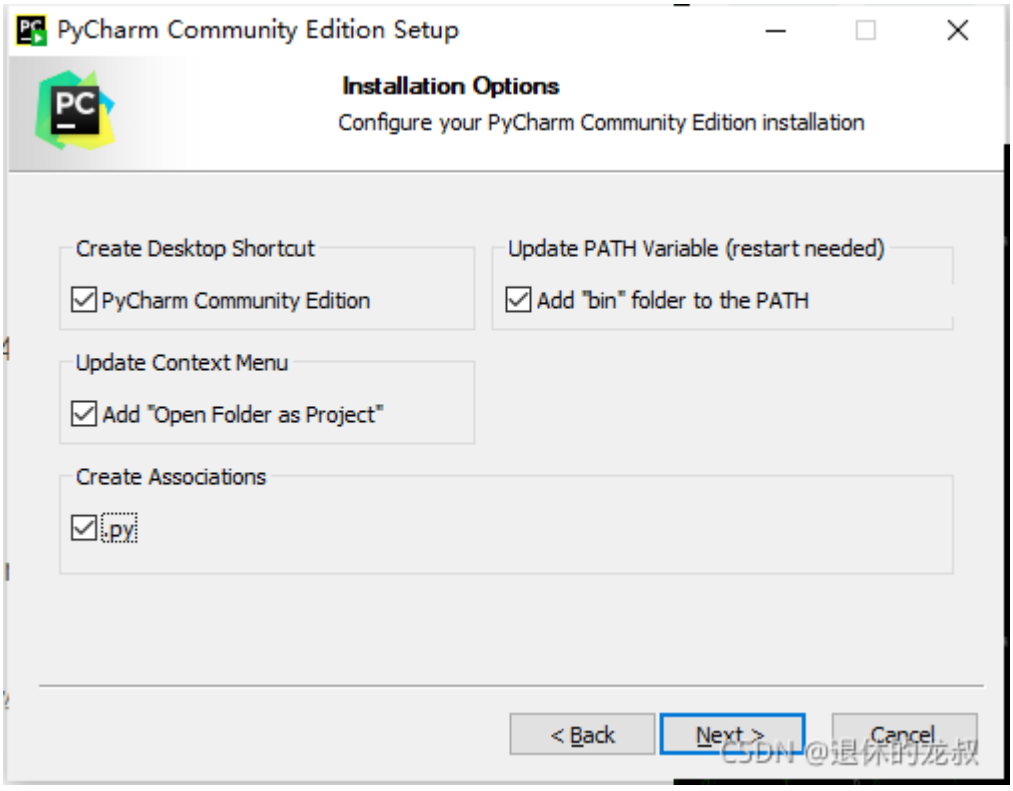
点击Next 。



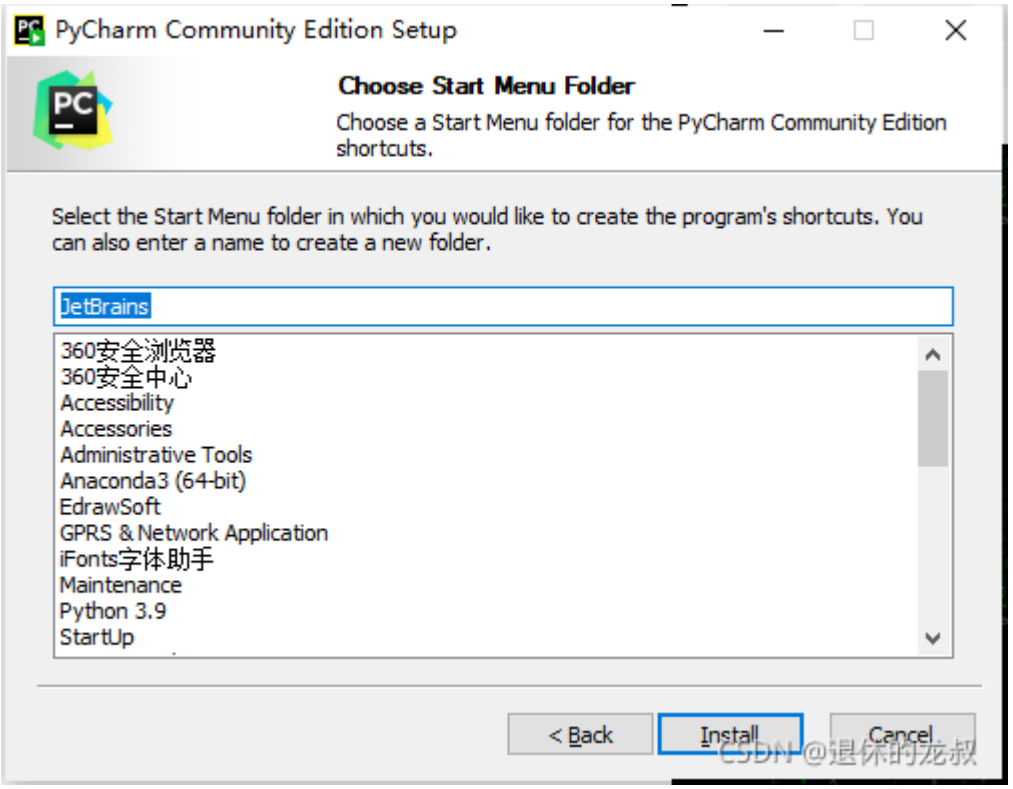
自定义安装路径，建议装在C盘之外的其他盘，点击 Next 。



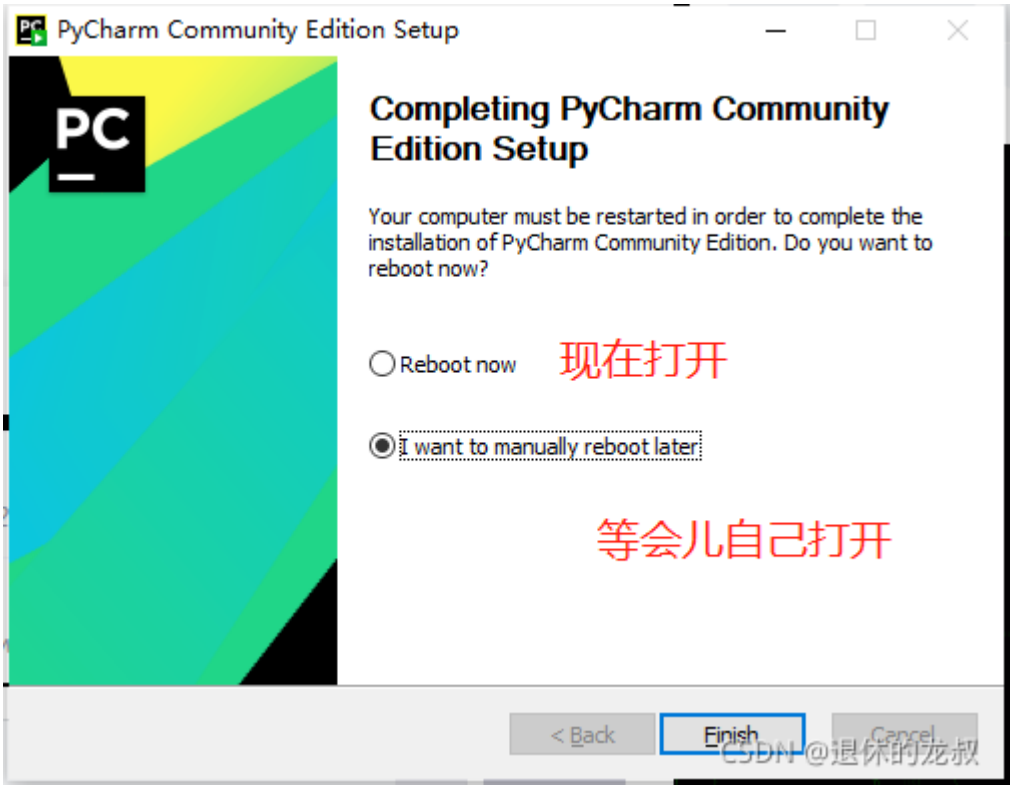
全部都勾选。



到了这里之后，默认即可，点击 Install 开始安装。



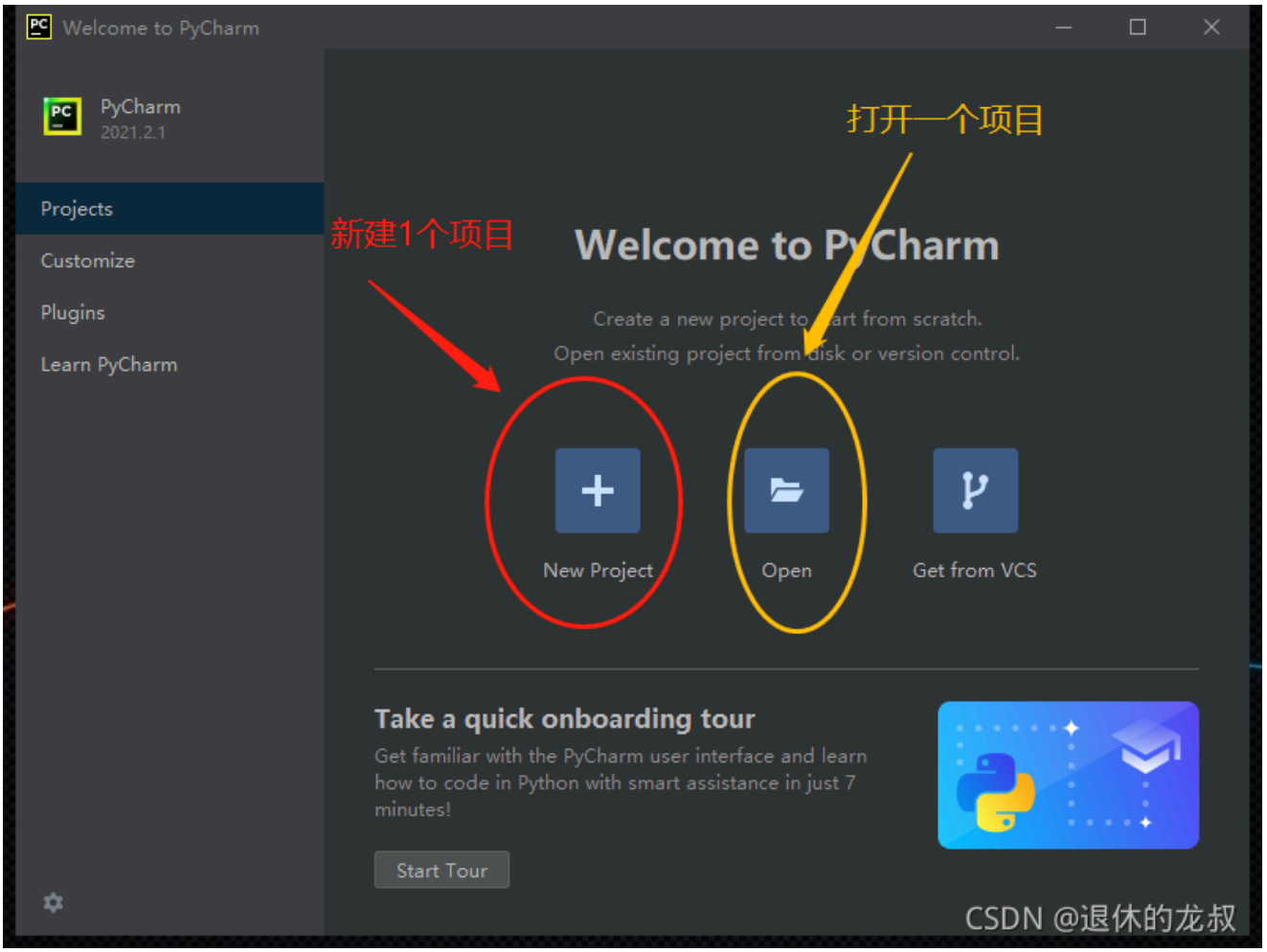
等待安装按成后，点击finish就装好了。



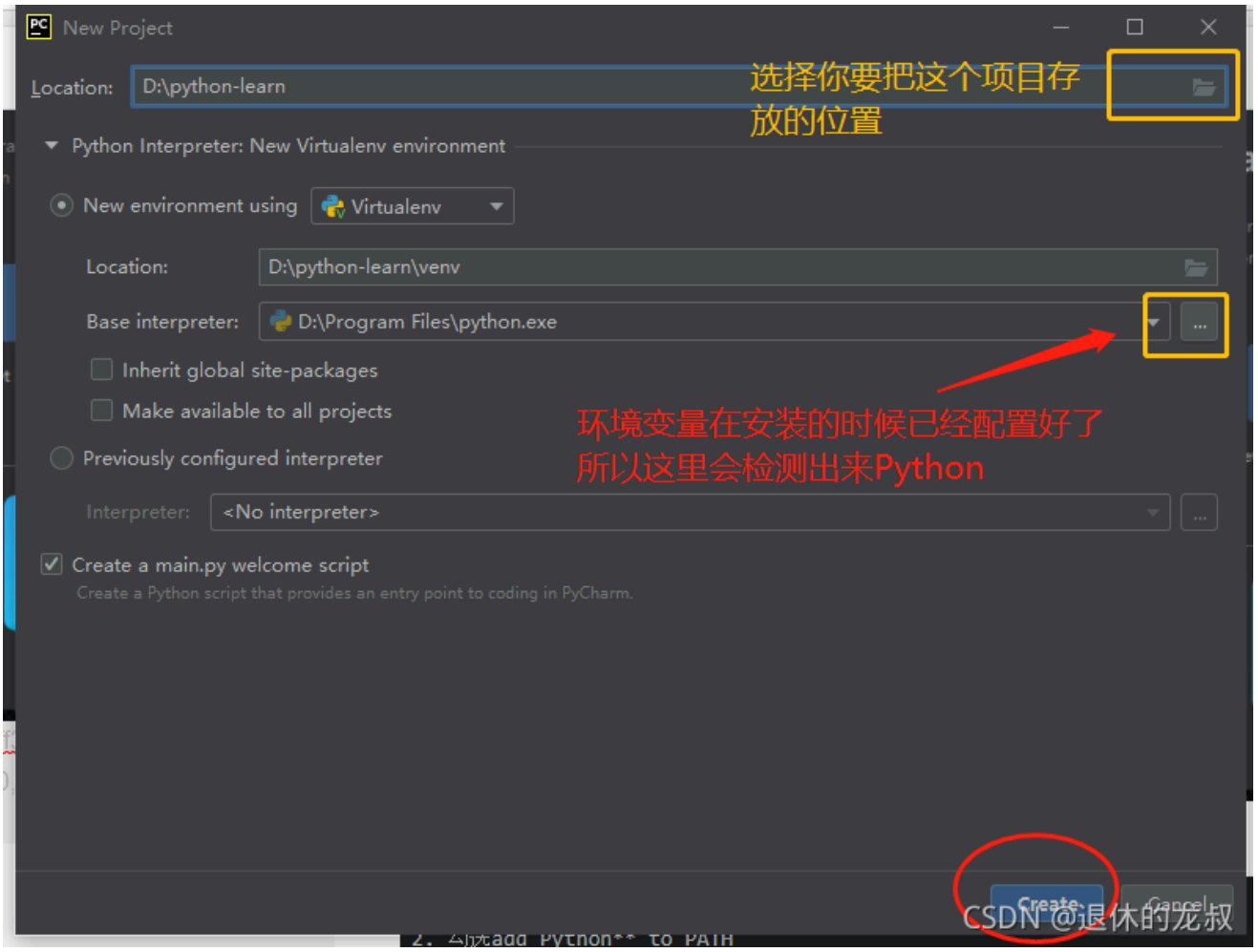
6.你的第一行代码

万事俱备，接下来我们开始写我们的第一行代码，鼎鼎大名的 hello world 。

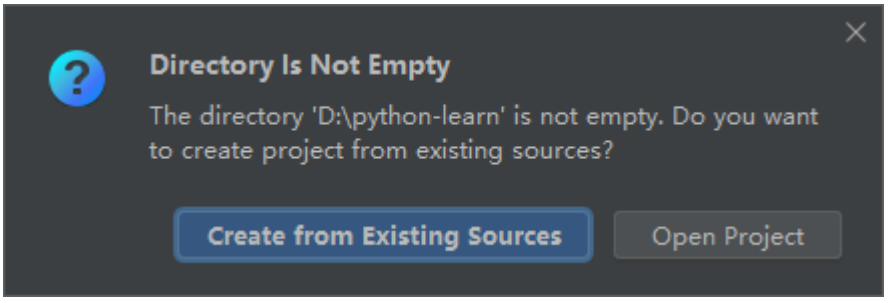
打开Pycharm，点击New Project 创建新项目，以后新建项目都可以这么操作，中间的Open 是打开项目，如果你有写好的项目就可以从这里打开。



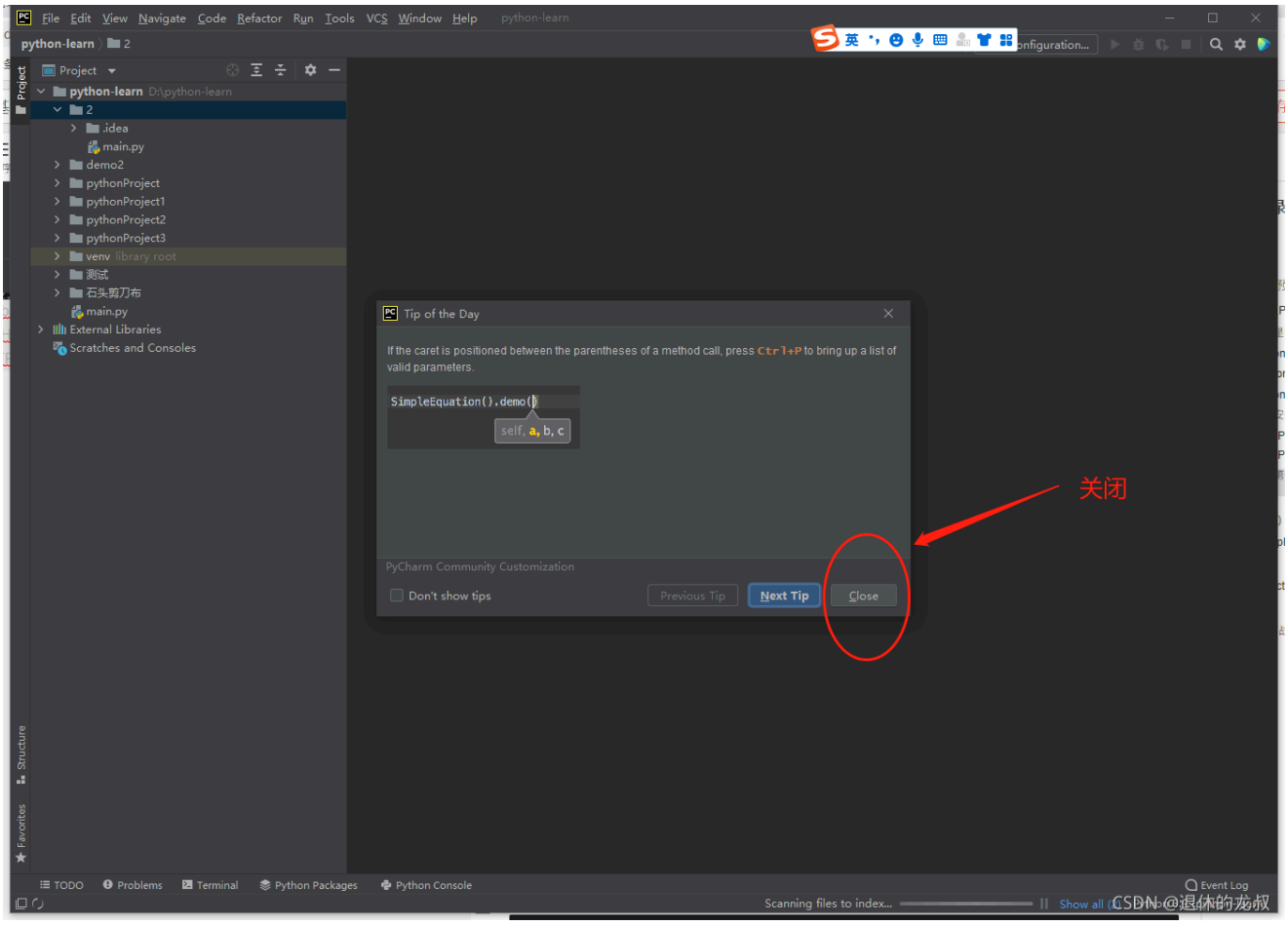
接下就是设置一下项目的存在位置，如果你的安装Pycharm的时候是按照我的步骤来的，Base interpreter 那里会自动检测出来Python，可以不做改到，直接点击左下角的Create 创建就可以了。



然后点击左边的。

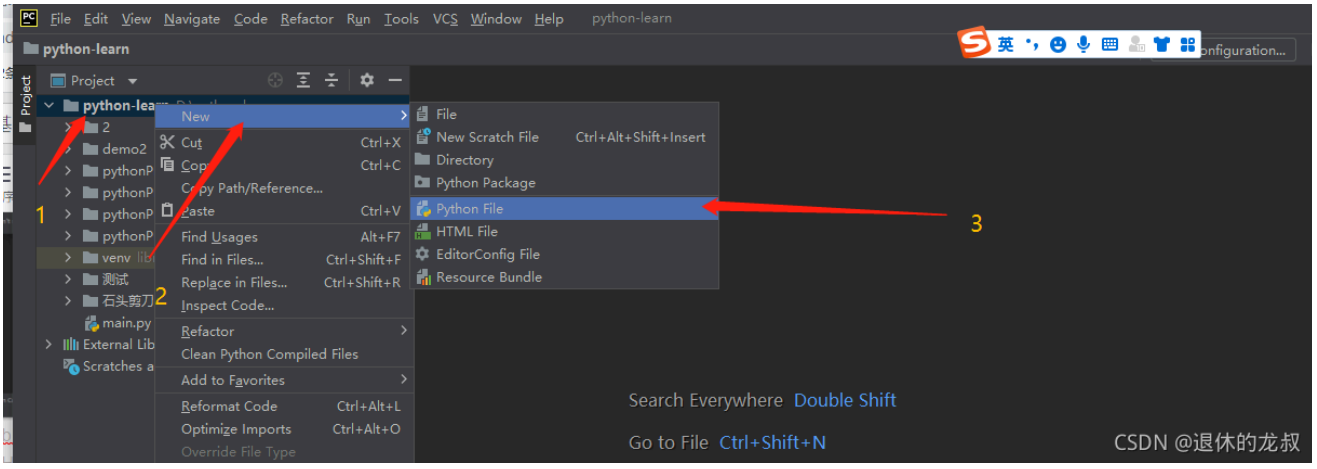


进入页面会弹出一个弹框，点击Close关闭就行。

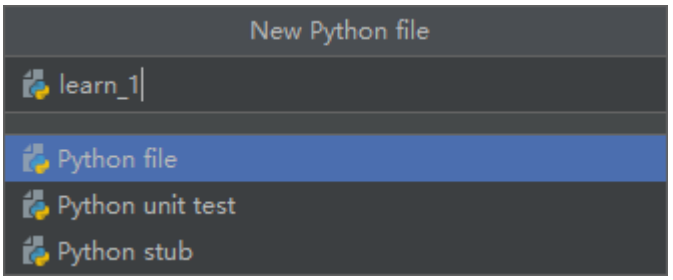


前面我们创建的是项目，是一个文件夹，来存放源码的，这里我们开始创建代码源文件。（我的电脑之前有过一些项目，所以底下有很多文件夹，可能跟你们不太一样。）

点击在创建好的项目python_learn，邮件弹出选项，选择New 来新建，选择Python File ，即创建Python源文件。



然后会提示你给你的文件取个名字，然后敲一下回车就完成了代码源文件的创建了。



开始写代码，这里需要用到的1个最常用的函数print() ，它是用来专门输出内容的，函数有它的语法，只要遵循语法来写代码，软件才会知道你要干什么。

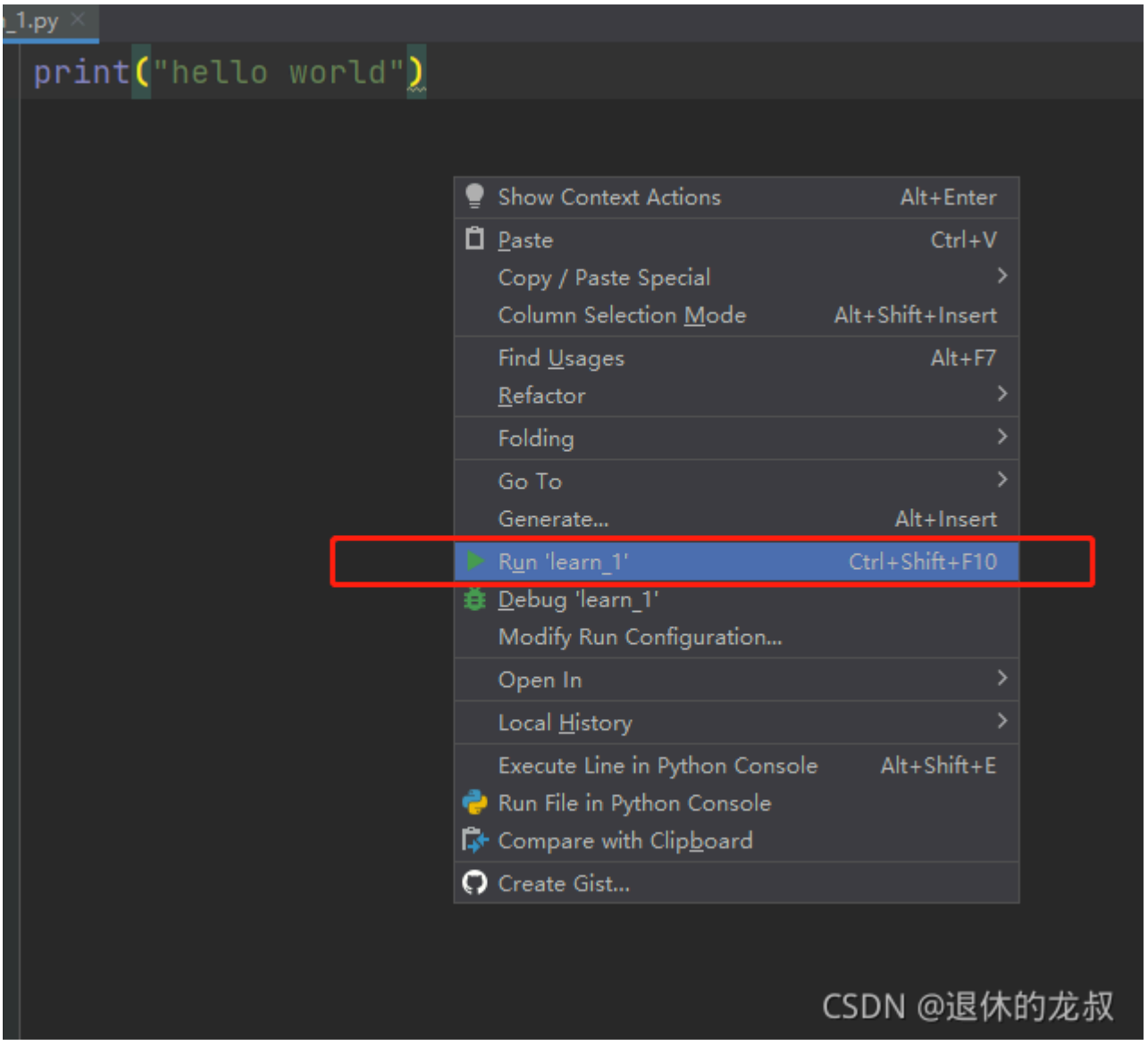
print的语法如下，默认自带换行。

```
print(要输出的内容)
python1
```

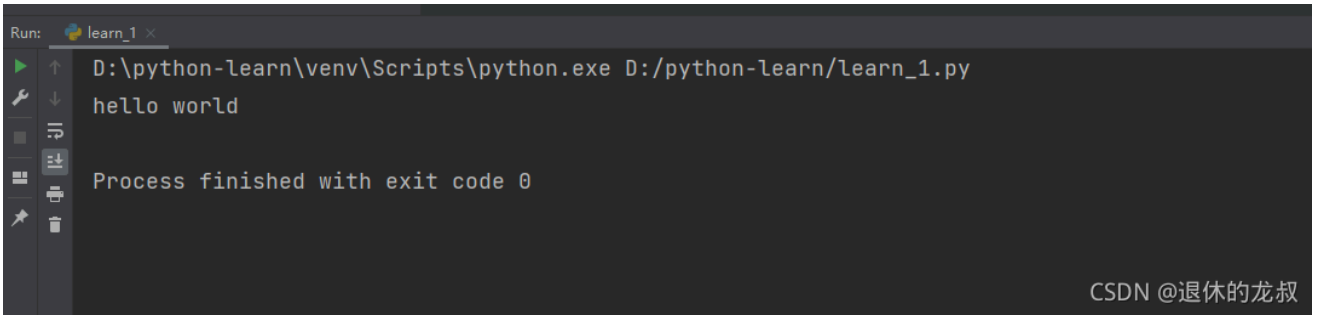
接下来我们就是要输出我们第一行代码了，hello world 是1串字符串，需要加上""才是正确的格式，不然会报错，关于字符串在后面我们会详细来讲，这里你只要知道得按照这个格式写代码才能达到我们想要的结果。

```
print("hello world")
python1
```

写完代码之后需要运行才出结果，可以右键弹出选项框，选择 Run'learn_1' 开始运行，Run是运行的意思，learn_1 是刚才我给这个源文件取的名字。



除了右键来运行，还可以直接用快捷键 Ctrl+Shift+F10，执行结果：



恭喜你，输出了你的第一行代码，正式开始Python之旅！

二、基础知识

在用Python写代码的之前，对Python的基础知识是必须要会的，不然你可能会寸步难行。基础知识包括输入输出、变量、数据类型、表达式、运算符这5个方面。

1.输入输出

Python有很多函数，后面我们会细讲，但这里先将两个最基本的函数：输入和输出。

输出函数print()，在前面我们已经用过了，语法就是：

```
print(要输出的内容)
python1
```

输入函数是 input()，功能是接收用户输入的内容，语法是：

```
input（输出的内容）
python1
```

举例：接收用户输入的密码并打印

```
n = input("请输入密码：")     #把输入内容赋给n，用 n 接收一下
print(n)     #打印n
python12
```

在Python里，“#” 表示注释，“#”后面的东西不会被执行。代码运行之后首先出现了“请输入密码:”，然后随意输入，比如输入123，执行结果：

```
提示信息
请输入密码：123
123
python123
```

成功接收用户输入的内容并打印出来。

2.变量

变量就是一个名字，需要先赋值在使用，变量要符合标识符(名字)的命名规范，这是硬性要求，标识符相当于名字，包括变量名、函数名、类名等等，

- **标识符的命名规范**

1. 合法的标识符:字母，数字(不能开头),下划线，py3可以用中文（不建议），py2不可以。
2. 大小写敏感。
3. 不能使用关键字和保留字。

> 关键字： if while for as import
> 保留字：input，print range

1. 没有长度限制。
2. 望文生义，看到名字就知道要表达的意思。
3. 大小写：

```
1. 包名：全小写，例如 time ；
  2. 类名：每个单词的首字母大写，其他的小写，简称大驼峰命名，例如 HelloWorld ；
  3. 变量名/函数名:第一个单词的首字母小写，后面的单词的首字母大写，简称小驼峰命名，例如 helloWorld ；
  4. 常量：全大写，例如 HELLO 。
1234
```

1. 其他命名方式，比如 hello_world 。

3.数据类型

- **数据类型分类**

  数据类型可分为以下6类：

```
（1） 整型：整数，英文名 int ，例如 5 的数据类型就是整型。
mar1
```

```
（2） 浮点型：小数，英文名 float ，例如 0.5 就是1个浮点型数据。

科学计数法，e表示乘以10几次方，例如 b=1e10 表示1*10的10次方。
mar123
```

```
（3） 字符串：英文str

表现形式有4种：'xs' 、 "xs" 、 """xsxs""" 、 ''''xxx'''

三引号有个特殊功能，表示注释，跟 # 一样的功能，例如：
mar12345
```

```
"""
xsx
```

```
xs
这里面的都是注释内容
"""
python12345
```

```
（4）布尔类型：英文bool，True为真，False为假；1表示真，0表示假。
mar1
```

```
（5）None 是一个单独的数据类型。
mar1
```

```
（6）列表、元组、字典、集合也是常见的数据类型。
mar1
```

- **类型转换**

在写代码的时候我们经常需要将不同的数据类型进行转换，主要的数据类型转换方法如下：

（1） 字符串转整型

方法是 int(str) ，字符串必须是数字，例如：

```
user = int('304200780')
print(user)
python12
```

执行结果：

```
304200780
python1
```

(2)浮点型转整型

方法是 int(float) ，例如：

```
f = 20.1
ff = int(f) #直接抹去小数部分
print(ff)
python123
```

运行结果：

```
20
python1
```

(3)字符串转浮点型

方法是 float(str) ，字符串必须是数字和，例如：

```
f = "203.4"
ff = float(f)
print(ff)
python123
```

运行结果：

```
203.4
python1
```

(4)整型转浮点型

方法是 float(int) ，例如：

```
f = 30
ff = float(f)  # 30.0
print(ff)
python123
```

运行结果：

```
30.0
python1
```

(5)浮点型转字符串

方法是 str(float) ，例如：

```
f = 30.5
ff = str(f)
print(type(ff).__name__)    #type()是获取数据类型函数
python123
```

这里先不管print()和type（）函数的嵌套，后面会将，只要知道它们是在输出 ff 的数据类型，运行结果：

```
str
python1
```

(6)整型转字符串
方法是 str(int) ，例如：

```
f = 30
ff = str(f)
print(type(ff).__name__)     #type()是获取数据类型函数
python123
```

- 获取类型信息
  常用的获取数据类型信息的函数有type()和isinstance()两个。

（1）type()
语法是 type(对象) ，返回的是对象的类型，前面我们也有用过，但是它是在内部返回的，如果你不输出它你是看不到的，所以经常会和输出函数print()嵌套使用。

例子：获取数据类型并输出。

```
f = 30
print(type(f))
python12
```

运行结果：

```
<class 'int'>
python1
```

class的意思是种类，可以看出 f 是 int 型的数据。

（2）isinstance()
isinstance() 常用来判断数据类型，它返回的是布尔值（True或False），语法是 isinstance(对象,class) 。

例子：判断30.5是不是整型。

```
f = 30.5
n = isinstance(f,int)    #用n来接收一下结果
print(n)
python123
```

运行结果：

```
False
python1
```

4.表达式

在Python当中，表达式是由数字、算符、数字分组符号（括号）、变量等对象的组合叫做表达式，表达式均有固定字面值，例如 “10+20”这个表达式的值为30，表达式 “10>30” 的值为 False 。

5.运算符

运算符可以分为很多4类。

（1）一般运算符

> +，-，*，/（真除法）,//（地板除，舍去小数部分）,%（取余数）,**（幂运算）

（2） 赋值运算符

> 常用赋值运算符是 = ，等号右边的值赋值等号左边

> 增强赋值运算符：+=，-=，*=，/=,%=,**=

例子：

```
a = 30
a+=10
print(a)
python123
```

执行结果：

```
40
python1
```

> 连续赋值：a=b=c=d=10

（3）布尔运算法

> == （等于），！ =（不等于） >= <= > <

（4） 逻辑运算符

> 主要有not、and和or三类，又称非、与和或
> and：前后都为真则为真
> or：有一个为真则为真
> not:非真，非假

例子：

```
a = 10
b = 20
c = 30
d = 40
n1 = a > b and a < c     #a>b为假，a<c为真，假与真为假
n2 = not a < c    #a<c为真，非真则为假
n3 = a > b or a < c      #a>b为假，a<c为真，假或真为真
print(n1,n2,n3)
python12345678
```

执行结果：

```
False False True
python1
```

三、流程控制

流程控制常用的是条件分支流程的if/else语句和循环控制的while语句。

1.条件分支流程

当达到某种条件的时候才会触发的代码。

（1）语法1

```
if 布尔表达式：#如果为真则执行内部的代码块
    代码块
python12
```

布尔表达式的结果只有两个，要么真，要么假，如果是真的，那么就执行if语句里面的代码块，否则就跳过不执行。

例子：

```
a = 10
b = 20
if a < b:
    print("真的")
if a > b:
    print("假的")

python1234567
```

执行结果：

```
真的
python1
```

从这里可以看出第一个if语句里面的布尔表达式（a<b）为真，所以执行了里面的代码块print(“真的”)，而第二个if语句里面的布尔表达式（a>b）是假的，所以里面的代码块没有被执行，所以不输出“假的”两字。

（2）语法2

常用的if/else语句，语法如下：

```
if 布尔表达式:
    代码块
else:
    代码块
python1234
```

判断的逻辑是如果布尔表达式为真，则执行if内部的代码块,如果为假则执行else内部的代码。

例子：

```
a = 10
b = 20
if a > b:
    a = 30
    print(a)

else:
    print(1111)
python12345678
```

执行结果：

```
1111
python1
```

（3）语法3

```
if 布尔表达式1:
    代码块
elif 布尔表达式2:
    代码块
elif 布尔表达式3:
    代码块
....
else:
    代码块
python123456789
```

逻辑是如果布尔表达式为真，则执行if内部的代码块,如果为假则执行else内部的代码，这个语法适用于多个连续条件判断。

例子：

```
s = int(input("请输入分数:"))

if 80 >= s >= 60:
    print("及格")
elif 80 < s <= 90:
    print("优秀")

elif 90 < s <= 100:
    print("非常优秀")

else:
    print("不及格")
python123456789101112
```

随意输入100以内的数字，不同区间内的数字结果是不一样的，例如92，执行结果为：

```
请输入分数:92
非常优秀
python12
```

```
10
python1
```

（4）语法4

这里可以将前面所讲的if/elif/else进行嵌套使用，来达到我们想要的目的。

例子

```
s = int(input("请输入分数:"))
if 80 >= s >= 60:
    print("及格")
elif 80 < s <= 90:
    print("优秀")
elif 90 < s <= 100:
    print("非常优秀")
else:
    print("不及格")
    if s > 50:
        print("你的分数在60分左右")
    else:
        print("你的分数低于50分")
python12345678910111213
```

随意输入数字，比如说55，执行结果：

```
请输入分数:55
不及格
你的分数在60分左右
python123
```

2.循环流程

在前面我们讲过的流程控制语句的功能还是比较有限，比如说只能执行1次，要执行多次就得多写几个，有点麻烦，所以我们需要学习循环流程里面的循环语句，它的作用就是重复运行某些代码。

（1）while循环

语法：

```
while 布尔表达式:
    代码块
```

```
python12
```

只要条件(布尔表达式)为真就执行里面的代码块。

举例：

```
while 4 < 5:
    s = int(input("请输入分数:"))
    if 80 >= s >= 60:
        print("及格")
    elif 80 < s <= 90:
        print("优秀")
    elif 90 < s <= 100:
        print("非常优秀")
    else:
        print("不及格")
        if s > 50:
            print("你的分数在60分左右")
        else:
            print("你的分数低于50分")
python1234567891011121314
```

运行之后可以多次输入分数，并且永不停息：

```
请输入分数:56
不及格
你的分数在60分左右
请输入分数:70
及格
请输入分数:
python123456
```

当然这里有1个弊端，代码执行后陷入了死循环,while里面的代码被一直执行，因为4<5永远为真。

所以这里我们可以改善一下代码，不让它永远执行，让它循环执行几次就可以了，这里可以用个变量来作为条件判断，把4<5改成a<5，同时让在while里面实现自加的功能，在while里面代码每执行1次，执行到它那行的时候它加1，这样执行2次while就会跳出来。

```
a = 3
while a < 5:
    s = int(input("请输入分数:"))
```

```
    if 80 >= s >= 60:
        print("及格")
    elif 80 < s <= 90:
        print("优秀")
    elif 90 < s <= 100:
        print("非常优秀")
    else:
        print("不及格")
        if s > 50:
            print("你的分数在60分左右")
        else:
            print("你的分数低于50分")
    a += 1
print(a)
print("while执行结束了")
python1234567891011121314151617181 9
```

执行结果：

```
请输入分数:55
不及格
你的分数在60分左右
请输入分数:65
及格
5
while执行结束了
python1234567
```

这里我再给大家举个例子来理解while循环的运用。比如说输入一个整数并计算各个位和，例如输入321，那么各个位之和则为6。

```
# 请输入一个整数，并计算各个位和 如：321=6

n = int(input("请输入一个整数:"))  # 将字符串转为整型

# sums累加器：m=10 m=10+5

sums = 0

while n != 0:  # 32 #3
    sums = sums + n % 10  # sums=1+2=3+3=6
    n = n // 10  # 32
print(sums)
python123456789101112
```

我输入的2345，执行结果：

```
请输入一个整数:2345
14
python12
```

（2）for循环

for循环和while循环都是循环语句，但不一样的点在于for循环是技术循环。

语法：

```
l=[3,2,1]
for 变量 in 可迭代对象:
    代码块
123
```

例子：

```
l=[3,2,1]
for n in l:
    print("1")
python123
```

执行：

```
1
1
1
python123
```

l是个列表，后面我们会讲，列表里面有3个元素，每执行一次for循环，列表里面的元素就会被赋值给n，直到列表里面没有了元素可赋值，则n就跳出了列表，此时的for循环就不成立了，不执行for里面的代码块。

（3）range

for循环经常会搭配range来使用，range是一个可迭代对象，range的语法如下：

```
range(start=0,stop,step=1)
1
```

start值的是开始下标。range序列里面的所有元素都有下标，第一个元素的下标是0，所以，默认是从0开始。

stop是结束位置。结束的位置下标为（元素个数-1），例如range里面有4个元素，那么结束下标最大为3,大于3则跳出range。

step是步长，如果step是2，那么每次会隔开1个元素，默认步长为1，即每个元素都会取到。

举例：

```
for i in range(8):    #可以不写star和step，但结束位置一定要写的
    print(i)
print("---------")
for i in range(10, 2, -2):
    print(i)
python12345
```

执行结果：

```
0
1
2
3
4
5
6
7
---------
10
8
6
4
python12345678910111213
```

通过第一个for循环可以看出，range()的第一个元素的下标是从0开始，而不是从1开始；range()可以不写开始下标和步长，但一定得有结束位置；第二个for循环可以看出步长可以为负数，用于递减。

（4）continue

continue的作用是跳过本次循环，后面的循环继续执行，例如：

```
for i in range(1, 10):
    if i == 5:
        continue
    print(i)
python1234
```

执行结果：

```
1
2
3
4
6
7
8
9
python12345678
```

很明显，i等于5的时候，for循环就跳过去了，本次不再执行里面的代码，重新回到了新的循环。

同样的，还有终止所有循环的功能，就是break，和continue是一样的用法，但效果是不一样的，我这里就不做举例了，大家可以去试一下就知道了。

四、列表(List)

列表是可以存放任何数据，包括整型，浮点型，字符串，布尔型等等，是常用的数据类型之一。

1.列表的创建

```
列表也是一个可迭代对象
1. 普通形式
        l = [1,2,3,4,5] ---整型列表
        l = ["a","b","c"] ---字符串列表
        l = [True,False,1>2,5<6]---布尔列表
2. 混合列表
        l = [1,2.5,"a",True]
3. 空列表
        l = []
123456789
```

- 从列表中如何获取数据(元素)

列表是有下标的，并且下标从0开始，元素是指列表中每个数据，例如l = [5,4,3,2,1] 里面有5个元素，但5的下标为0,1的下标为4。

```
1.获取当个数据(元素)
    语法：
        变量[下标] #获取下标所对应的值
123
```

例如获取l列表中下标为1的元素：

```
l = [1, 2, 3]  # 下标/索引：0开始
print(l[1])
python12
```

执行结果为：

```
2
python1
```

```
2. 列表的遍历
列表的遍历是把列表里面的所有元素都获取1遍。
12
```

例如把[1,2,3,4,5]里面的元素都获取并输出一遍：

```
l = [1,2,3,4,5]
for i in l:
    print(i, end=" ")
python123
```

执行结果为：

```
1 2 3 4 5
python1
```

```
3. 交换数据
对指定下标的元素进行数据交换。
12
```

例如把[1, 2, 3, 4, 5]里面的下标为第2和第3的元素进行数据交换：

```
l = [1, 2, 3, 4, 5]  # 下标/索引：0开始
l[2], l[3] = l[3], l[2]
```

```
print(l)
python123
```

执行结果：

```
[1, 2, 4, 3, 5]
python1
```

2.添加元素

添加元素的方法常用的有以下3个：

序号	函数	说明
1	append()	向列表中添加对象，并添加到末尾
2	extend(可迭代对象)	将可迭代对象中数据分别添加到列表中，并添加到末尾
3	insert(下标,对象)	向指定下标位置添加对象

统一用法是：

```
变量.函数
#例如 n. append(对象)
12
```

例子:讲列表[6,7]添加到另一个列表[1,2,3,4,5]中

```
l = [1, 2, 3, 4, 5]
l.extend([6, 7])
print(l)
python123
```

执行结果：

```
[1, 2, 3, 4, 5, 6, 7]
python1
```

3.删除元素

删除列表中的元素的常用方法主要有：

4	clear()	清空列表
5	pop()	删除下标指定的元素，如果不加下标则删除最后一个元素
6	remove(对象)	删除指定的对象
7	del	删除变量或指定下表的值

例子：删除列表[1, 2, 3, 4, 5]中4这个元素。

```
l = [1, 2, 3, 4, 5]
l.remove(4)
print(l)
python123
```

执行结果为：

```
[1, 2, 3, 5]
python1
```

4. 修改元素

修改列表中的元素方法其实很简单，直接用这个方式就可以了：

```
变量[下标]=新值
1
```

例如：

```
l = [1, 2, 3, 4, 5]
l[2]=6
print(l)
python123
```

执行结果：

```
[1, 2, 6, 4, 5]
python1
```

5.列表高级特性的

- **切片操作**

  切片，顾名思义就是把1个列表切分为多个列表，语法如下：

```
变量[起始下标:结束下标]      #结束下标取不到
1
```

  例如

```
l = [1, 2, 3, 4, 5]
print(l[0:4])
python12
```

  执行结果：

```
[1, 2, 3, 4]
python1
```

  做切片操作时要注意以下几个点：

```
①如果下标从0开始可以省略不写，例如 n = l[:4]
②如果结束下标取的是最后一个元素，可以省略不写，例如 n = l[3:]
③如果列表中的元素都要，开始和结束下标都可以省略，例如 n = l[:]
④n = l[:-1] 表示从0开始 - 到数二个元素
1234
```

- **列表的进阶操作**
  方法是 n = l[开始:结束:步长] ，这个方法既可以正向去操作列表，也可以反向去操作列表,例如:

```
l = [1, 2, 3, 4, 5]
n = l[-1:-3:-1]
print(n)
python123
```

  执行结果：

```
[5, 4]
python1
```

- 列表的一些操作符

1. 比较运算符
   列表之间进行比较，以相同下标进行比较，从小到大进行比较，如果值相同则比较下一组元素，如果不同直接出结果，例如：

```
l = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  # 下标/索引：0开始
l2 = [2, 3, 4, 6]
print(l < l2)  # True
python123
```

   执行结果：

```
True
python1
```

1. 逻辑运算符

   逻辑运算符and not or 跟比较运算符相似，返回结果都是布尔值（True/False），这里可以去自己尝试一下。

1. 拼接运算符

   拼接运算符是 + ，常用来进行两个列表拼接，例如：

```
l = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  # 下标/索引：0开始
l2 = [2, 3, 4, 6]
print(l + l2)
python123
```

   执行结果：

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 2, 3, 4, 6]
python1
```

1. 重复操作符

   重复操作符为 * ，后面常跟数字，表示将列表里面的元素重复复制几遍,例如：

```
l2 = [2, 3, 4, 6]
print(l2*2)
python12
```

执行结果：

```
[2, 3, 4, 6, 2, 3, 4, 6]
python1
```

1. 成员关系操作符

成员关系操作符主要有 in和not in，用来判断元素是否在列表中，返回结果是布尔值，例如：

```
l = [2, 3, 4, 6]
print(5 not in l)     #输出“5不在列表l中”这句话的真假
python12
```

执行结果：

```
True
python1
```

- 列表的其他方法

列表常用的其他方法还有以下几个：

8	copy()	浅拷贝
9	count(对象)	返回对象在列表中出现的次数
10	index(value,开始下标,结束下标)	元素出现的第一次下标位置，也可自定义范围
11	reverse()	原地翻转
12	sort (key=None, reverse=False)	快速排序，默认从小到大排序，key:算法
13	len()	获取列表的长度（元素）

CSDN @退休的龙叔

例如：将列表里面的所有元素进行翻转

```
l = [2, 3, 4, 6]
l.reverse()
print(l)
python123
```

执行结果：

```
[6, 4, 3, 2]
python1
```

- 冒泡排序法

冒泡排序是列表里面比较有名的排序方法之一，例如将列表[5,4,3,2,1]里面的所有元素用冒泡排序的思想进行从小到大排序。

```
l = [6, 5, 4, 3, 2, 1]
for i in range(1, len(l)):  # 1,5 #循环4大次  4
    for j in range(len(l) - i):
        if l[j] > l[j + 1]:
            l[j], l[j + 1] = l[j + 1], l[j]

print(l)
python1234567
```

执行结果：

```
[1, 2, 3, 4, 5, 6]
python1
```

- 选择排序

选择排序是让列表中的元素，固定一个元素和其他元素进行比较，不符合条件互换位置。

```
l = [6, 5, 4, 3, 2, 1]
for i in range(0, len(l) - 1):  # 0,4
    for j in range(i + 1, len(l)):  # 1,5
        if l[i] > l[j]:
            l[i], l[j] = l[j], l[i]

print(l)
python1234567
```

执行结果：

```
[1, 2, 3, 4, 5, 6]
python1
```

- 二维列表

列表里面还能存放多个列表，由比如列表 [[1,2,3],[4,5,6],[7,8,9]] ，它是由两个维度的列表组成，1个维度是它本身，另一个维度是[1,2,3],[4,5,6],[7,8,9] 这三个列表，所以构成了二位列表。

对于二位列表，语法是：

```
变量[外层列表下标][内层列表的下标]
1
```

例如输出二位列表中的第一个列表里面的下标为1的元素:

```
l = [[1,2,3],[4,5,6],[7,8,9]]
print(l[0][1]) #2
python12
```

执行结果为:

```
2
python1
```

在这里我们可以看得出来，二位列表里面的外层列表也有下标，下标也是从开始。

我们再来对二位列表进行遍历：

```
l = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for i in l:
    for j in i:
        print(j)
python1234
```

执行结果：

```
1
2
3
4
5
6
7
8
9
python123456789
```

五、元组（tuple）

元祖也是Python中常见的数据类型之一，它可以用来存放任何数据类型，但它也有它的特点：

```
1. 不能修改，不可变类型
2. 用（）的形式
3. 元组也是可迭代对象
4. 元组是有序的，下标操作,支持切面操作[:]
1234
```

1.元组的创建及访问

```
1. 创建：
    直接创建，例如  t = (1,2,3,4,5)
2. 访问：
    t[下标]  #获取元素
3. 切片操作：
    t[:] 返回元组
123456
```

例子创建元组并对元组进行元素访问和切片操作：

```
t = (1, 2, 3, 4, 5)
print(t[2]) #输出下标为2的元组元素
n = t[2:4]  #切片，范围是下标2~4
print(n)
python1234
```

执行结果为：

```
3
(3, 4)
python12
```

2.修改和删除

前面有说过元组是不可变类型，不能修改，但是可以通过将元组转换成列表的形式进行修改和删除等操作，最后再将列表转换成元组，完成元组的修改和删除。

例如：修改元组中的元素

```
t = (1, 2, 3, 4, 5)
l = list(t) #将元组转换成列表
print(l)    #输出列表
l[2] = 6    #列表指定元素的修改
print(l)    #输出新列表
t = tuple(l)    #列表转换成元组
print(t)
python1234567
```

执行结果：

```
[1, 2, 3, 4, 5]
[1, 2, 6, 4, 5]
(1, 2, 6, 4, 5)
python123
```

删除元组中的元素可用 del t[下标] 方法，前提还是一样的先将元组转换成列表，例如：

```
t = (1, 2, 3, 4, 5)
l = list(t) #将元组转换成列表
print(l)    #输出列表
del l[2]    #列表指定元素的删除
print(l)    #输出新列表
t = tuple(l)    #列表转换成元组
print(t)
python1234567
```

执行结果：

```
[1, 2, 3, 4, 5]
[1, 2, 4, 5]
(1, 2, 4, 5)
python123
```

3.元组的操作符

元组同样也有着操作符，方法跟列表的操作符是一样的。

```
1. 比较运算符
        < > >= <= == !=
```

```
2. 逻辑运算符
    and not or
3. 拼接运算符
    +
4. 重复操作符
        *
5. 成员关系操作符
        in    not in
        判断元素是否在列表中
        print(5 not in l)
123456789101112
```

例子：判断5在不在元组里面。

```
t = (1, 2, 3, 4, 5)
print(5 not in t)    #5不在元组里面
python12
```

执行结果：

```
False
python1
```

4.元组的方法

元组的方法和前面所介绍的列表的方法是一样的，这里给大家回顾一下列表的常用方法：

3. 列表		
序号	函数	说明
1	append()	向列表中添加对象，并添加到末尾
2	extend(可迭代对象)	将可迭代对象中数据分别添加到列表中，并添加到末尾
3	insert(下标,对象)	向指定下标位置添加对象
4	clear()	清空列表
5	pop()	删除下标指定的元素，如果不加下标则删除最后一个元素
6	remove(对象)	删除指定的对象
7	del	删除变量或指定下表的值
8	copy()	浅拷贝
9	count(对象)	返回对象在列表中出现的次数
10	index(value,开始下标,结束下标)	元素出现的第一次下标位置，也可自定义范围
11	reverse()	原地翻转
12	sort (key=None, reverse=False)	快速排序，默认从小到大排序，key:算法
13	len()	获取列表的长度（元素）

@退休的龙叔

这些方法的使用还是遵循列表的方法时所讲的 变量名.方法 。除了上面的这些，还有2个方法值得新增：

```
1. count(value)
统计某个值出现的次数，value是指定的值

2. index(value,[start],[stop])
返回value在元组中出现的下标位置（第一次出现的下标）
12345
```

例子：统计3在元组（1,2,3,4,5,3）中第一次出现的下标。

```
t = (1, 2, 3, 4, 5,3)
l1 = t.index(3, 0, 6)
print(l1)
python123
```

执行结果：

```
2
python1
```

六、字符串

在Python中，字符和字符串没有区别。可能有些同学学过其他的语言，例如Java，在Java中，单引号'a'表示字符'a'，双引号"abc"表示字符串"abc"，但在Python当中，它们没有区别，都统称字符串。

1.字符串的特点

字符串拥有以下特点：

```
1. 字符串不可变类型
2. 字符串是可迭代对象
3. 字符串支持索引和切片操作
4. 支持操作符；
        拼接：+
        重复操作符：*
        比较运算符：  > < <= >= == !=
        逻辑运算符：not and or
        成员关系：  in    not in
123456789
```

2.字符串的方法

字符串的常用方法有以下这些：

序号	函数	说明
1	capitalize()	把字符串的第一个字符改为大写，后面的小写
2	casefold()	把整个字符串都小写
3	encode()	编码 str--bytes（二进制字符串）
4	decode()	解码
5	count(sub, start, stop)	返回字符（sub）出现的次数，star：开始下标，stop:结束下标
6	find(sub, start, stop)	返回sub第一次出现的下标，查不到返回-1
7	index(sub, start, stop)	返回sub第一次出现的下标
8	upper()	将字符串转为大写
9	lower()	将字符串转为小写

CSDN @退休的龙叔

例子：将字符串 “hello world” 的第一个字母大写。

```
a = "hello world"
b = a.capitalize()
print(b)
python123
```

执行结果：

```
Hello world
python1
```

3.格式化输出

格式化输出是指将字符串按照某种格式进行输出,常用的方法有 format 和 $% 。

```
1. format 语法1：用数字占位（下标）
1
```

例子:

```
"{0} 嘿嘿".format("Python")
a = 100
s = "{0}{1}{2} 嘿嘿"
s2 = s.format(a, "JAVA", "C++")
print(s2)
python12345
```

```
2.format 语法2：{} 占位
1
```

例子：

```
a = 100
s = "{}{}{} 嘿嘿"
s2 = s.format(a, "JAVA", "C++", "C# ")
print(s2)
python1234
```

执行结果：

```
100JAVAC++ 嘿嘿
python1
```

```
3.format 语法3：{} 占位用字母占位
1
```

例子：

```
s = "{a}{b}{c} 嘿嘿"
s2 = s.format(b="JAVA", a="C++", c="C# ")
print(s2)
python123
```

执行结果：

```
C++JAVAC#  嘿嘿
python1
```

```
4.%s
语法为 “%s”%（值） ，最常用的参数可以是任意值。
12
```

例子：用%s 结合循环语句输出九九乘法表

```
for i in range(1, 10):
    for j in range(1, i + 1):
        print("%s * %s = %s" % (i, j, i * j), end="\t")
    print()

python12345
```

执行结果：

```
1 * 1 = 1
2 * 1 = 2    2 * 2 = 4
3 * 1 = 3    3 * 2 = 6    3 * 3 = 9
4 * 1 = 4    4 * 2 = 8    4 * 3 = 12    4 * 4 = 16
5 * 1 = 5    5 * 2 = 10    5 * 3 = 15    5 * 4 = 20    5 * 5 = 25
6 * 1 = 6    6 * 2 = 12    6 * 3 = 18    6 * 4 = 24    6 * 5 = 30    6
* 6 = 36
7 * 1 = 7    7 * 2 = 14    7 * 3 = 21    7 * 4 = 28    7 * 5 = 35    7
```

```
* 6 = 42    7 * 7 = 49
8 * 1 = 8    8 * 2 = 16    8 * 3 = 24    8 * 4 = 32    8 * 5 = 40    8
* 6 = 48    8 * 7 = 56    8 * 8 = 64
9 * 1 = 9    9 * 2 = 18    9 * 3 = 27    9 * 4 = 36    9 * 5 = 45    9
* 6 = 54    9 * 7 = 63    9 * 8 = 72    9 * 9 = 81
python123456789
```

4.转义字符

```
1. “\n” ： 换行符
2. “\'”:单引号
3. “\“”:双引号
4. "\\" : \
1234
```

在这里值得注意的是 \ ，它有很多比较巧的运用，比如说当你在同一行要写的东西比较多的时候，视觉上不是很好看,可以用反斜杠来进行视觉上的换行，但上一行和下一行在逻辑上是一样的，例如：

```
a = "sxsxsxsxsxsxsxs\
        xsxsxsxs\
        xsx"
print(a)

a = 1 + 2 + 3 \
    + 4
print(a)
python12345678
```

执行结果：

```
sxsxsxsxsxsxsxs        xsxsxsxs        xsx
10
python12
```

七、字典（dict）

字典是用来存储数据的，字典中的数据以映射关系存储。

1.字典的特点

```
1. 字典是Python中唯一的映射类型
2. 字典是无序的
3. 字典是可迭代对象
4. 字典的构成
    键：key
    值：value
    映射：键映射值
    键-值：键值对，又叫 项
12345678
```

2.创建字典

```
1. 直接创建
        语法：  d = {}      #空字典
        例如：  d = {"name":"不良人","apple":"苹果"}
123
```

```
2. dict()
        例如：d = dict()     #空字典
12
```

```
3. dict(可迭代对象)
1
```

例如：

```
d3 = dict([("one",1),("two",2)])
        print(d3)
python12
```

执行结果：

```
{'one': 1, 'two': 2}
python1
```

这就是一个元组，one是键，1是值，‘one’: 1 是键值对。

```
4. dict(**kwargs)
1
```

例如：

```
d4 = dict(a=3, b=4)
print(d4)
python12
```

执行结果：

```
{'a': 3, 'b': 4}
python1
```

3.字典访问的

```
1. 基本形式：
        变量名[键名] #键所对应的值
12
```

例如：

```
d = {"name": "小黑"}
print(d["name"])
python12
```

执行结果：

```
小黑
python1
```

```
2. 添加一个键值对
        变量名[键名]=值
3. 修改一个键值对的值
        变量名[键名]=值
1234
```

4.字典的方法

字典常用的方法主要有以上一些：

6. 字典		
序号	函数	说明
1	clear()	清空字典
2	copy()	浅拷贝
3	fromkeys(可迭代对象，value=None)	根据可迭代对象中的元素去创建字典
4	get(key,[d])	获取键所对应的值，key是键，d是提示信息
5	items()	将字典中的键值对封装成元组并放到类集合中
6	pop(key,[d])	根据键删除字典中的键值对，*key*是键，*d*是提示信息
7	values()	返回字典中的值（类集合对象）
8	popitem()	删除某个键值对
9	values()	返回字典中的值（类集合对象）

CSDN@逸休的龙叔

例子：返回指定字典中的所有值

```
d2 = {'c': 3, 'hehe': 100, 'b': 2, 'a': 1}
c = d2.values()
print(list(c))
python123
```

执行结果：

```
[3, 100, 2, 1]
python1
```

5.补充

关于字典，有一些小点给大家做个补充：

```
1. 字典可以使用for循环
    for i in d2:
        print(i) #键，不包含值
2. 输出一个键值对
    for i in d2.items():
```

```
        print(i)
3. 成员关系操作符
        in/not in
        只能查询键
123456789
```

八、函数

函数是由一组代码组成，完成某个特定的功能。

1.创建和使用

创建函数的语法如下：

```
def 函数名(参数):
    代码块（函数的实现/函数体）
12
```

参数相当于变量,参数可以为1个或者多个，用逗号隔开，还可以没有参数，等于无参；代码块是函数的实现，又叫函数体。

函数的调用

```
函数名(参数)
1
```

2.函数的运行机制

函数的运行遵循以下机制：

```
1. 从函数调用开始执行
2. 通过函数名字找到函数定义的位置（创建函数的位置）
3. 执行函数体
4. 执行完毕之后，返回到函数的调用处
1234
```

3.函数的使用

举例：用函数对古诗《登鹳雀楼》进行逐行打印，并对每一行下面增加由多个相同符号组成的分割线。

```
def fun2(a, n):
    print(a * n)

# 打印古诗
print("白日依山尽")
fun2("+", 5)
print("黄河入海流")
fun2("-", 10)
print("欲穷千里目")
fun2("*", 15)
print("更上一层楼")
fun2("/", 20)
python12345678910111213
```

执行结果：

```
白日依山尽
+++++
黄河入海流
----------
欲穷千里目
***************
更上一层楼
////////////////////
python12345678
```

4.函数的特点

通过上面的例子可以发现，函数具有以下特点：

```
1. 避免了代码的冗余
2. 提高了代码的可维护性
3. 提高了代码的可重用性
4. 提高了代码的灵活性
1234
```

5.函数的参数

函数的参数首先要明白以下三个概念：

```
1. 形式参数：形参
        在定义函数的时候传递的参数
2. 实际参数：实参
        在调用函数时传递的参数
3. 无参
        没有任何参数
123456
```

```python
def fun2(a, n):
    print(a * n)


# 打印古诗
print("白日依山尽")
fun2("+", 5)
print("黄河入海流")
fun2("-", 10)
print("欲穷千里目")
fun2("*", 15)
print("更上一层楼")
fun2("/", 20)
```

a和n是形参

+和5是实参

CSDN @退休的龙叔

```
位置参数：
    实参的位置和形参一一对应，不能多也不能少。
12
```

```
def fun4(a, b, c, d):
    print(a, b, c)
fun4(10, 20, 30)
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
Traceback (most recent call last):
  File "D:\python-learn\learn_1.py", line 3, in <module>
    fun4(10, 20, 30)
TypeError: fun4() missing 1 required positional argument: 'd'

Process finished with exit code 1
```

少传了1个值给形参d，只传了3个值

CSDN @退休的龙叔

关键字参数：

用形参的名字作为关键字来指定具体传递的值，则在函数调用时，前后顺序将不受影响。

12

```
def fun4(a, b, c):
    print(a, b, c)

fun4(b=10, a=20, c=30)
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
20 10 30

Process finished with exit code 0
```

CSDN @退休的龙叔

位置参数和关键字参数混用：

当位置参数和关键字参数混用时，位置参数在前

12

```
def fun4(a, b, c):
    print(a, b, c)

fun4(a=10, 20, c=30)
```

关键字参数不能在前面
只要把 a=10 换成 10 ,则变为位置参数在前，结果是没有报错的

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
  File "D:\python-learn\learn_1.py", line 4
    fun4(a=10, 20, c=30)
                       ^
SyntaxError: positional argument follows keyword argument

Process finished with exit code 1
```

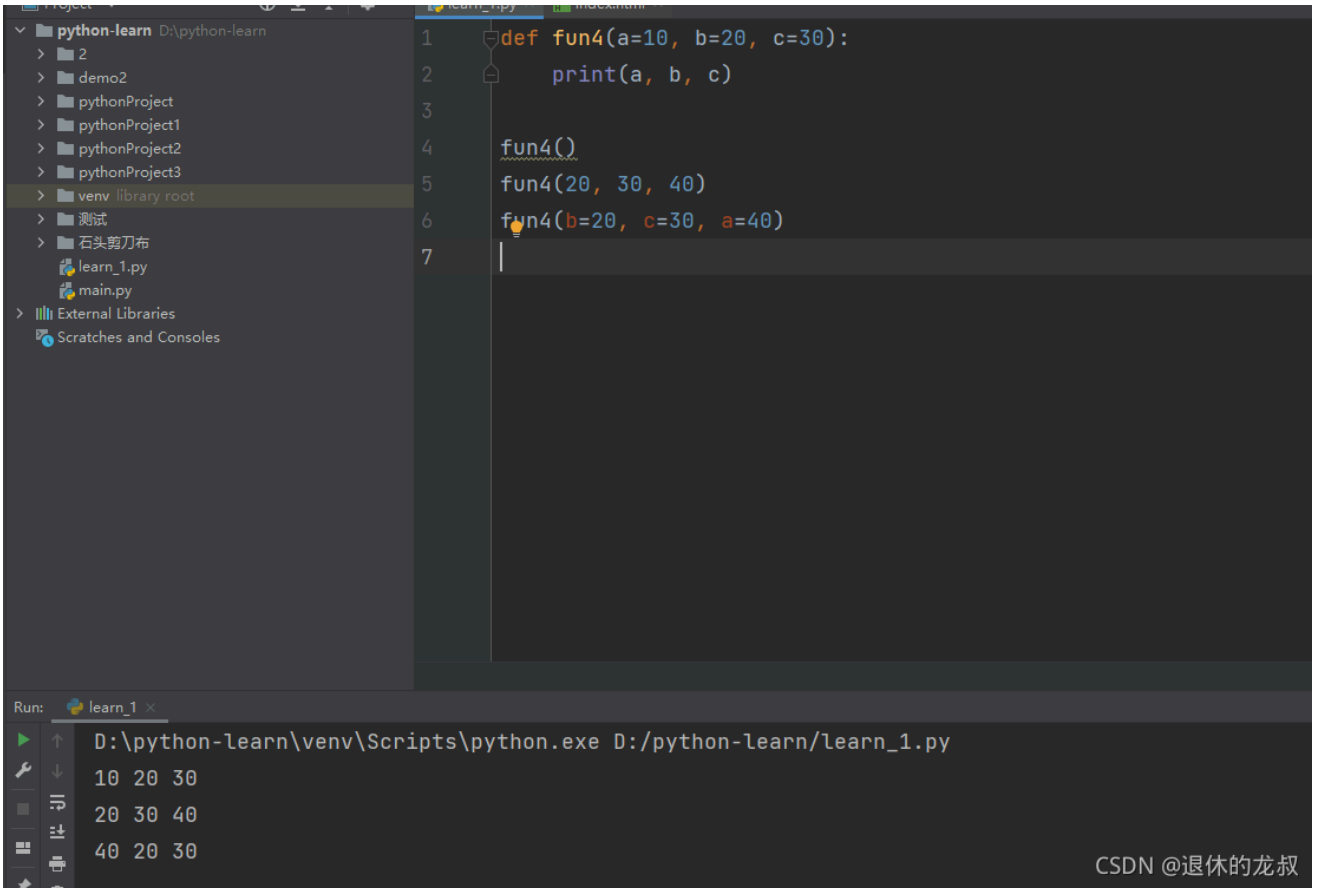
CSDN @退休的龙叔

默认参数：

给了默认值的参数--形参

如果传递了新的值，那么默认值就被覆盖了

123

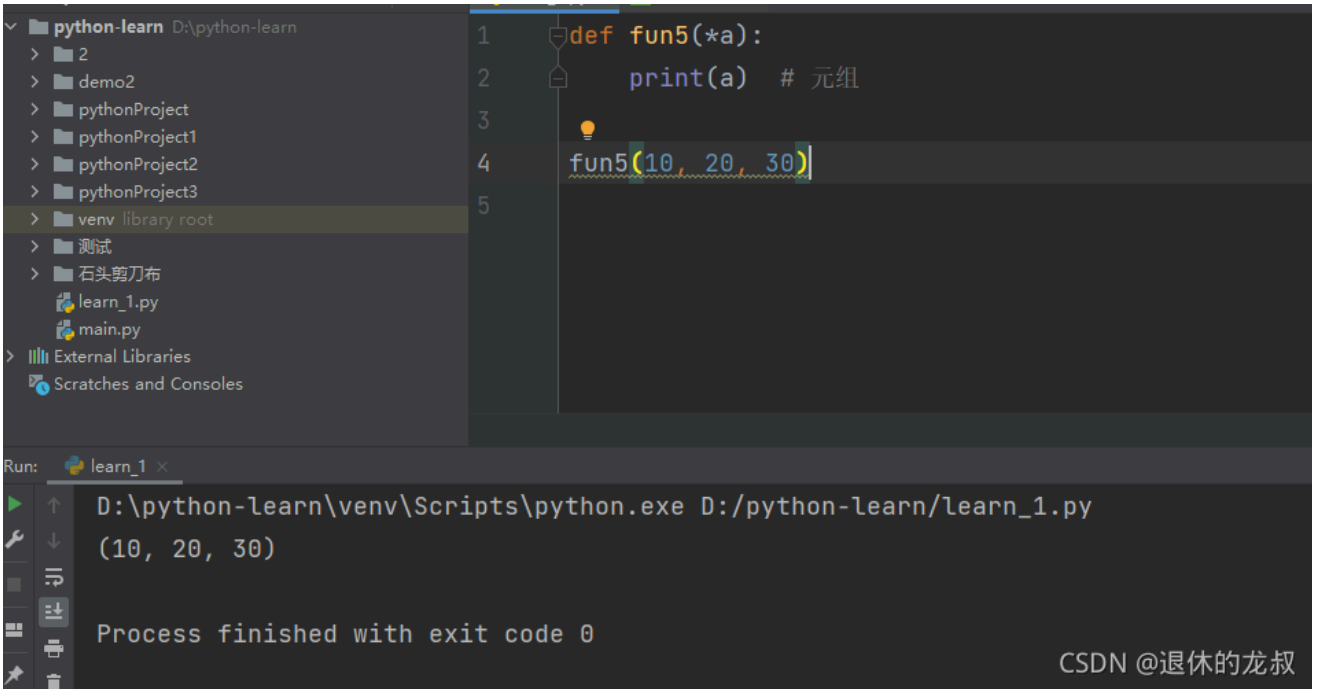


可变成参数：

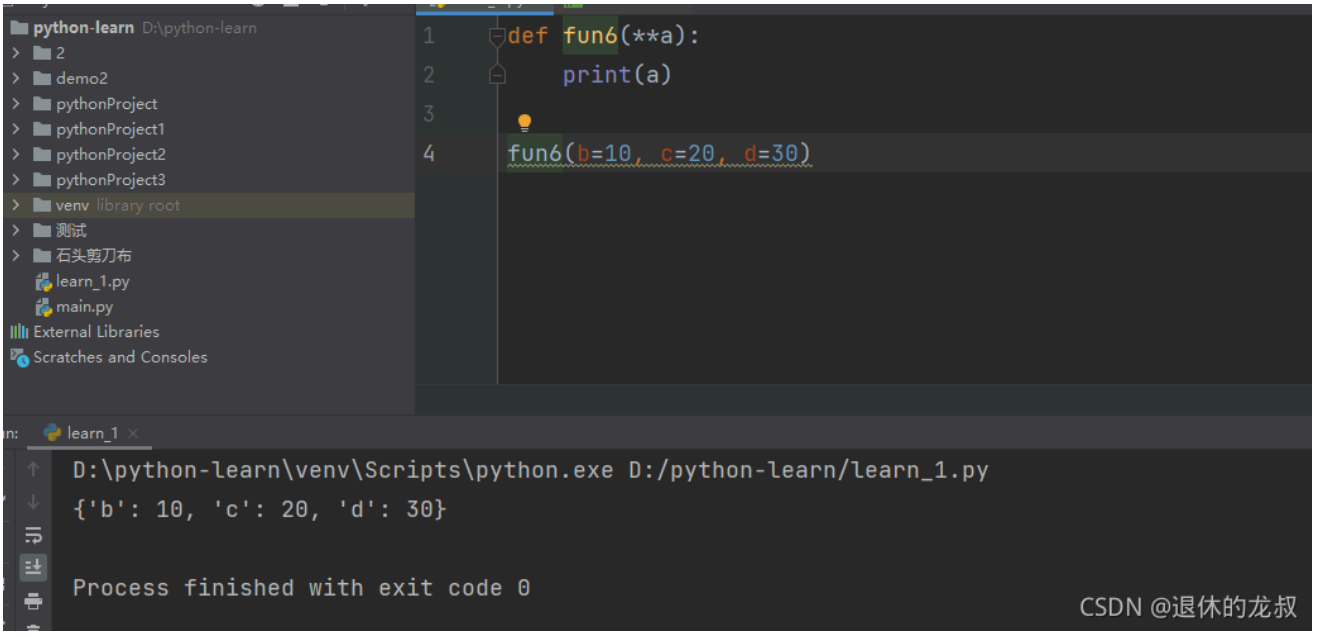
```
def 函数名(*a)
```

本质上封装成了元组

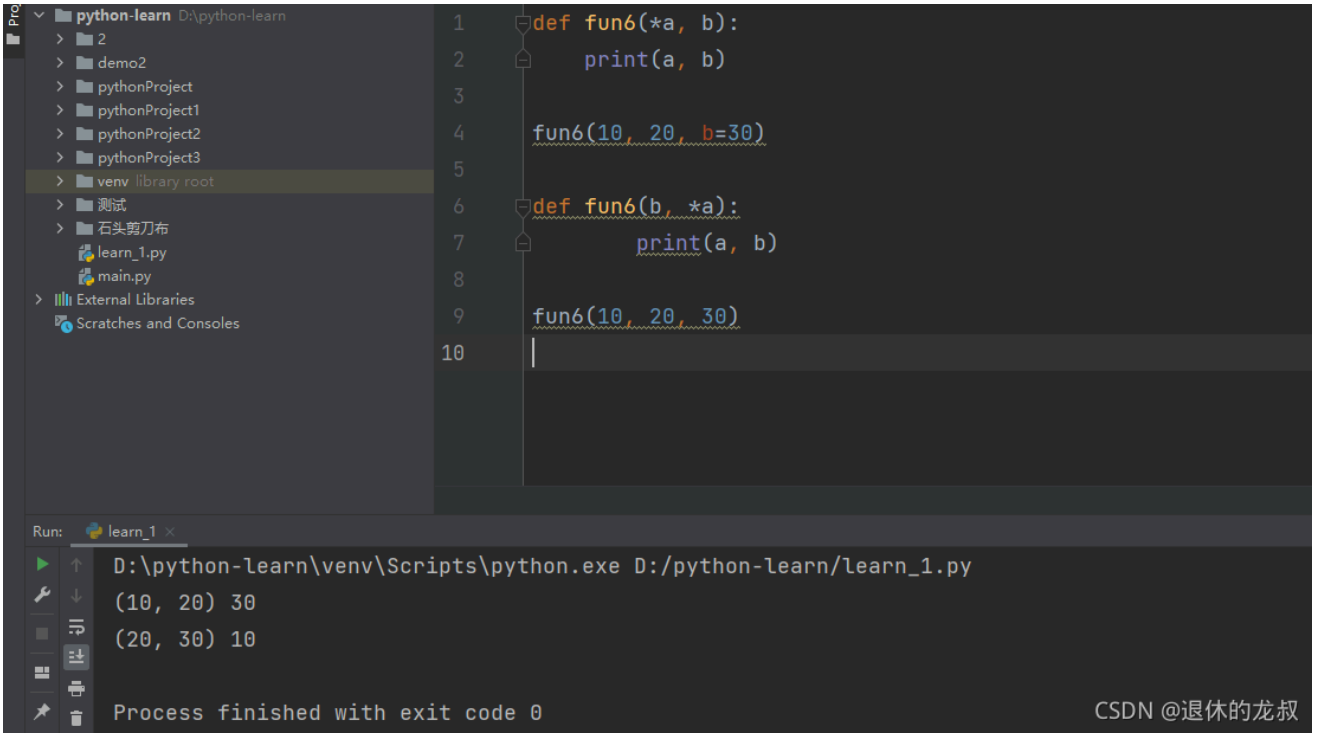
123



```
可变成参数：
    def 函数名(**kwargs)
        将参数封装成了字典
123
```




```
可变成参数和位置参数混用的时候：可变参数优先
1
```



6.函数的文档

写代码的时候我们经常需要写文档，前面有提过#和三引号可以进行代码的注释，但在这里要介绍一种新的方法，也是写代码时常用的函数文档书写格式：在函数体的第一行用 “”” “”” 进行文档说明，这是标准化的函数文档书写。

例如：

```
def fun6(b, *a):
    """
    fun6是个函数
    a:可变成参数
    b:位置参数
    """
    print(a, b)

fun6(10, 20, 30)
```

CSDN @退休的龙叔

拥有函数说明文档之后，就可以获取函数的文档内容，方法是：

```
函数名.__doc__
1
```

例如：

```python
def fun6(b, *a):
    """
    fun6是个函数
    a:可变成参数
    b:位置参数
    """
    print(a, b)

fun6(10, 20, 30)
print(fun6.__doc__)
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
(20, 30) 10

    fun6是个函数
    a:可变成参数
    b:位置参数
```

CSDN @退休的龙叔

除此之外，还可以用 help(函数名) 的方式进行函数文档的查看，例如：

```python
def fun6(b, *a):
    """
    fun6是个函数
    a:可变成参数
    b:位置参数
    """
    print(a, b)

help(fun6)
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
Help on function fun6 in module __main__:

fun6(b, *a)
    fun6是个函数
    a:可变成参数
    b:位置参数


Process finished with exit code 0
```

CSDN @退休的龙叔

7.函数的返回值

```
关键字：return
返回值谁调用就返回给谁
1. 任何函数都有返回值
2. 如果不写return ，也会默认加一个return None
3. 如果写return ，不写返回值 也会默认加一个None
4. 可以返回任何数据类型
5. return后面的代码不在执行，代表着函数的结束
1234567
```

8.函数的变量的作用域

首先需要明白两个概念：局部变量和全局变量。

```
1. 局部变量
        定义在函数内部的变量
        先赋值在使用
        从定义开始到包含他的代码结束
        在外部无法访问
2. 全局变量
        1. 定义在源文件的开头
        2. 作用范围：整个文件
        3. 局部变量和全局变量发生命名冲突时，以局部变量优先
123456789
```

```
3.global
        声明全局变量
12
```

```python
def fun1():
    global b
    b = 100
    print(b)

fun1()
print(b)
```

局部变量b被global声明后
在函数外部也能使用

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
100
100

Process finished with exit code 0
```

CSDN @退休的龙叔

4. nonlocal

声明的是局部变量

12

```python
def fun1():
    b = 100
    def fun2():
        nonlocal b
        b = 300
        print(b)
    fun2()
    print(b)
fun1()
print(b)
```

声明之后 b在函数内变成了300
如果不声明，则在这里输出的是100

b还是局部变量

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
Traceback (most recent call last):
  File "D:\python-learn\learn_1.py", line 10, in <module>
    print(b)
NameError: name 'b' is not defined
300
300

Process finished with exit code 1
```

CSDN @退休的龙叔

9.内嵌函数

定义在函数内部的函数叫做内嵌函数（或者叫内部函数），内部函数的作用范围：从定义开始到包含给他的代码块结束在内部函数中不能进行a+=1,a=a+1这样的操作，解决方案是nonlocal 和global。

例如：

10.闭包

闭包是函数式编程的重要语法结构。

编程范式：一种编程范式，对代码进行提炼和抽象概括，使得重用性更高

如果内部函数调用了外部函数的局部变量，并外部函数返回内部函数的函数对象（函数名），例如：

```python
def fun1():
    a=100
    def fun2():
        print(a)
        return "哈哈"
    return fun2

print(fun1()())
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
100
哈哈

Process finished with exit code 0
```

CSDN @退休的龙叔

这就形成了闭包，闭包的作用是可以传递更少的形参，可以传递更多的实参—更加安全，间接性的访问内部函数，例如：

```python
def fun1(b):
    a = 100
    print(b)

    def fun2(c):
        print(c)
        return "哈哈"

    return fun2


print(fun1(200)(300))
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
200
300
哈哈

Process finished with exit code 0
```

CSDN @退休的龙叔

当然了，闭包是有条件的：

```
1. 必须是一个内嵌函数
2. 外部函数必须返回内部函数的函数对象
3. 内部函数必须使用外部函数的局部变量
123
```

11. lambda表达式

```
1. 匿名函数
        没有名字的函数
2. 使用时机：
        只使用一次的时候
3. 语法：
        关键字：  lambda
        lambda 参数1，参数2:返回值
4. lambda的书写形式更加的简洁，优雅
        l = lambda x:x
        print(l(100))
5. lambda的优先级最低
1234567891011
```

九、综合性练手项目

学完了这些基础你觉得你的水平怎么样？是不是就这么飘过来的？说实话，学完这些基础你能做的东西没有多少的，好玩的东西都是入门之后才能做出来的。

但是在龙叔这里，综合性项目还是有的，就根据我上面所讲的循环语句、列表、函数等基础知识，可以尝试写一个综合性的项目来检验你的知识，可以尝试写1个简单的名片管理系统。

（一）系统需求

1.程序启动，显示名片管理系统欢迎界面，并显示功能菜单

```
**************************************************
欢迎使用【名片管理系统】V1.0

1. 新建名片
2. 显示全部
3. 查询名片
```

```
0. 退出系统
**************************************************
python123456789
```

2.用户用数字选择不同的功能
3.根据功能选择，执行不同的功能
4.用户名片需要记录用户的 姓名、电话、QQ、邮件
5.如果查询到指定的名片，用户可以选择 修改 或者 删除 名片

（二）效果预览

大概了解了这个操作系统的需求之后，我们先不忙着看代码，我们先来看看代码运行后的效果，这就是我们要做的东西：

这里我不放视频了，放视频的文章一点开就会自动播放，这点有点无语，尤其是我这种长篇文章，你找半天找不到视频在哪？哈哈哈...所以还是给大家放视频演示的链接吧！

视频效果链接： https://www.bilibili.com/video/BV1aZ4y1F7n6/

下次我会专门出一篇讲这个名片管理系统项目的文章，现在你们先暂时自己尝试着写写吧，源码私聊找我要也可以的。

【】

虽然头发越来越少了.....
但只要还有人点赞、关注和收藏
我就会继续更新出很哇塞的内容！

微信名片

内容来源：csdn.net

作者昵称：退休的龙叔

原文链接：https://sslljy.blog.csdn.net/article/details/120217658

作者主页：https://sslljy.blog.csdn.net

打赏作者

¥1 ¥2 ¥4 ¥6 ¥10 ¥20

扫码支付： ¥1

获取中

扫码支付

您的余额不足，请更换扫码支付或 充值

打赏作者

实付 元

使用余额支付

点击重新获取

扫码支付

钱包余额 0

抵扣说明：

1.余额是钱包充值的虚拟货币，按照1:1的比例进行支付金额的抵扣。
2.余额无法直接购买下载，可以购买VIP、付费专栏及课程。

余额充值

举报

AI 搜索 智能体 AI 编程 AI 作业助手

隐藏侧栏 搜索

程序员都在用的中文IT技术交流社区

专业的中文 IT 技术社区，与千万技术人共成长

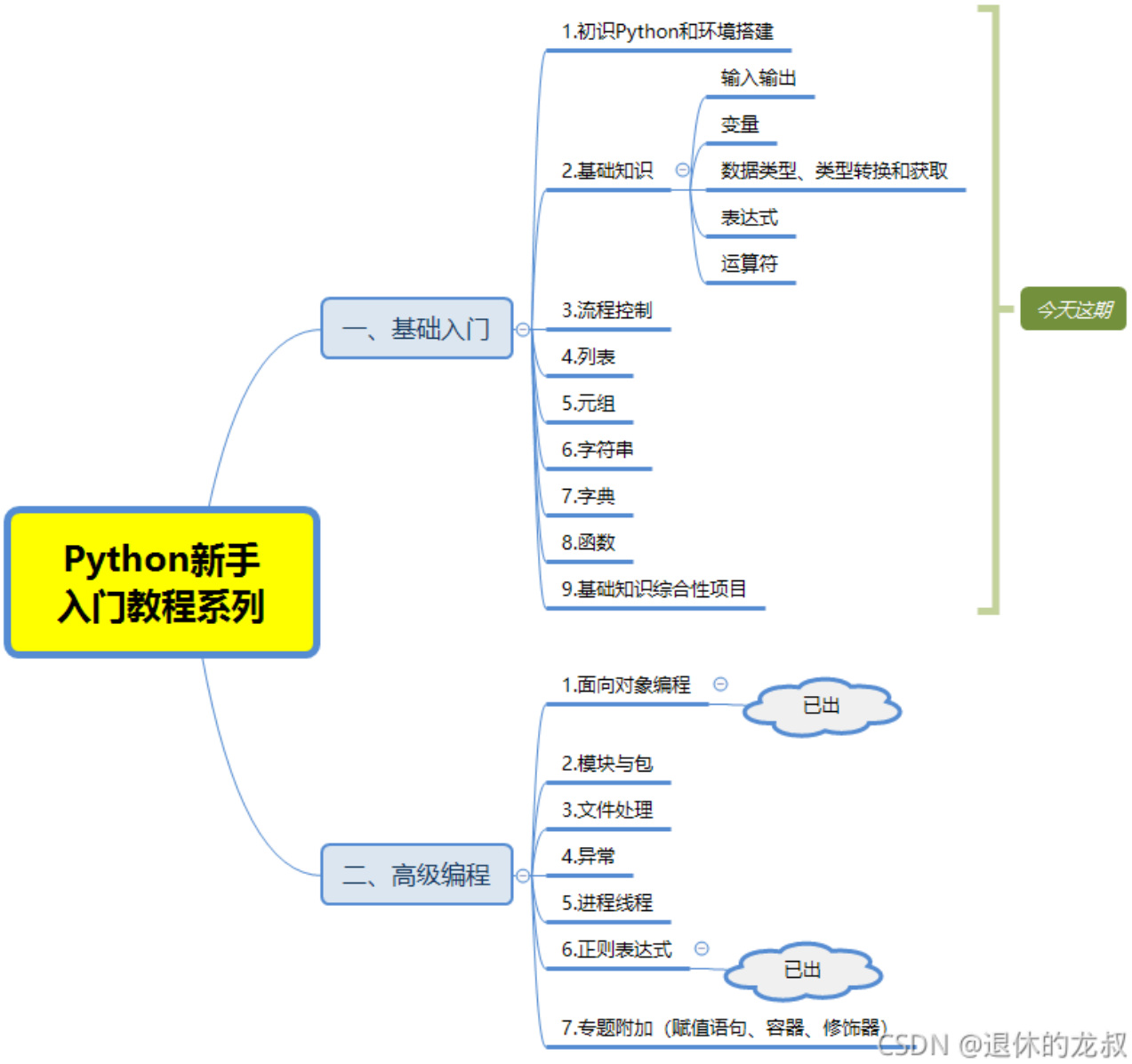
关注【CSDN】视频号，行业资讯、技术分享精彩不断，直播好礼送不停！

客服 返回顶部

微信名片

微信 ID：长按可保存，到微信“扫一扫”添加好友可领 微信扫码添加好友或搜索 ID

复制微信 ID


Python新手
入门教程系列
一、基础入门
1.初识Python和环境搭建
2.基础知识
输入输出
变量
数据类型、类型转换和获取
表达式
运算符
3.流程控制
4.列表
5.元组
6.字符串
7.字典
8.函数
9.基础知识综合性项目
今天这期
二、高级编程
1.面向对象编程
已出
2.模块与包
3.文件处理
4.异常
5.进程线程
6.正则表达式
已出
7.专题附加（赋值语句、容器、修饰器）
CSDN @退休的龙叔

hello world
CSDN @退休的龙叔

Guido van Rossum
@gvanrossum

I decided that retirement was boring and have joined the Developer Division at Microsoft. To do what? Too many options to say! But it'll make using Python better for sure (and not just on Windows :-). There's lots of open source here. Watch this space.

Python创始人Guido van Rossum（吉多·范罗苏姆）在推文中表示：“我个人觉得退休生活实在是太无聊了，所以我加入了微软开发部门。你问我到底要干啥？能干的事情太多了！不过，我会让大家在使用Python这款软件的时候，觉得更为方便（不仅仅是在windows系统上）。毕竟有不少开源的东西。”

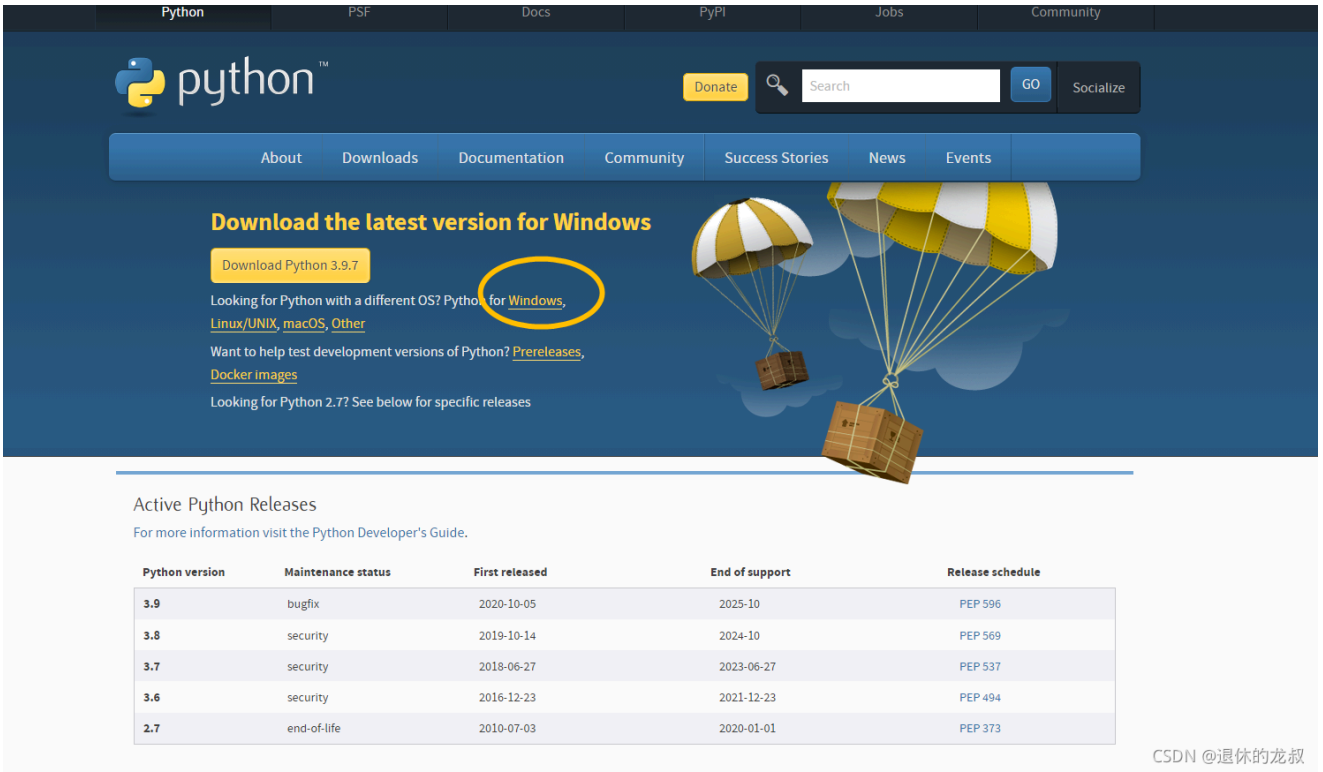
在这条推文下，以下国外网友也给出了一些很有意思的评论：

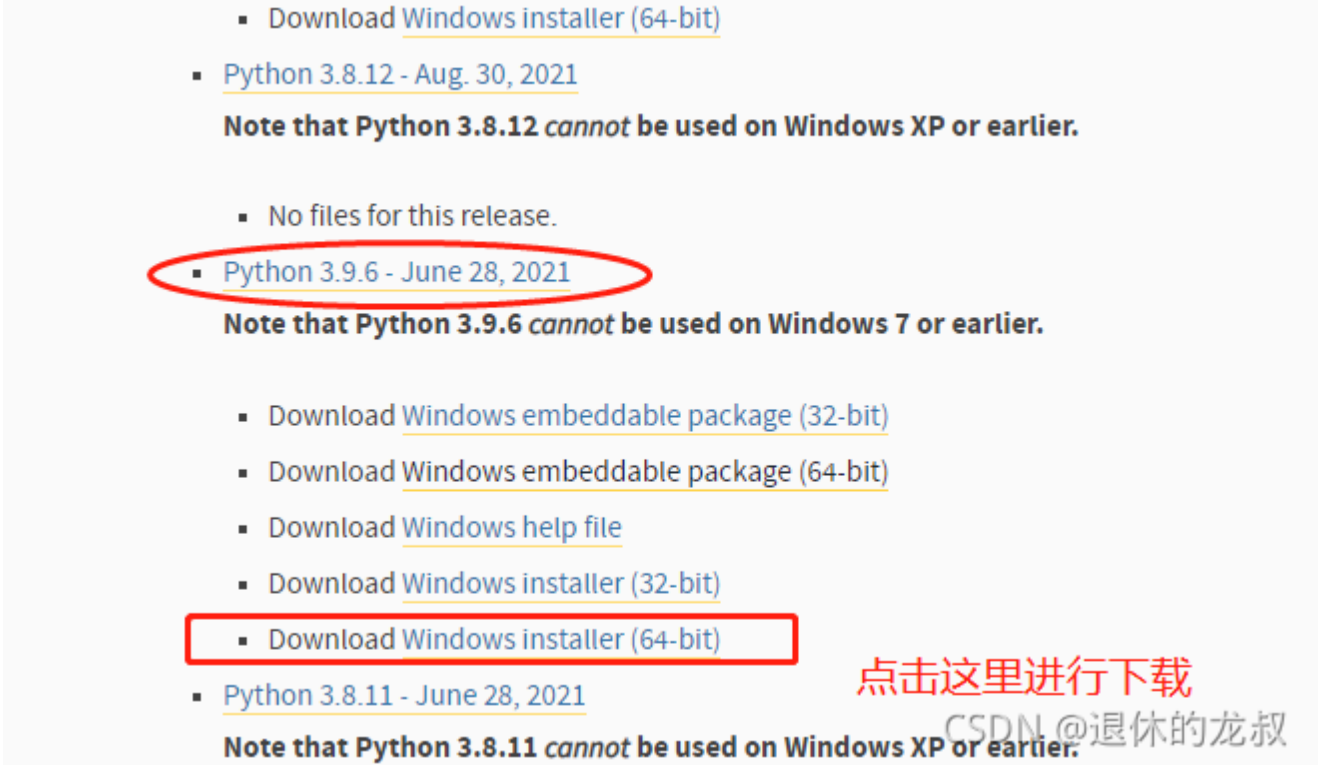
“麻烦大佬把Python加入Excel。”

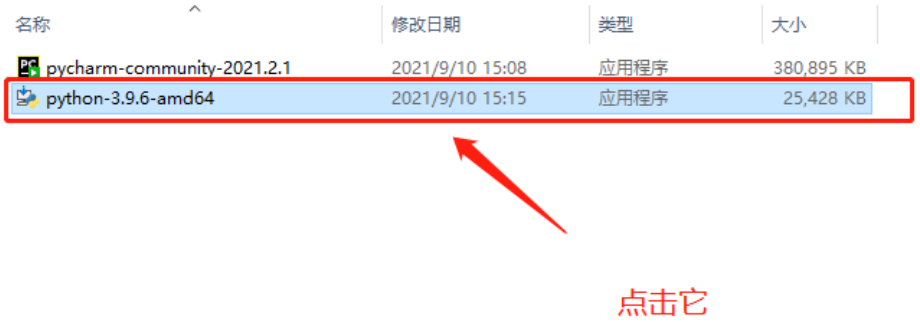
Diptarshi Chakraborty @DiptarshiC · 14小时
回复 @gvanrossum
Add python to excel

1 3 53

CSDN @退休的龙叔

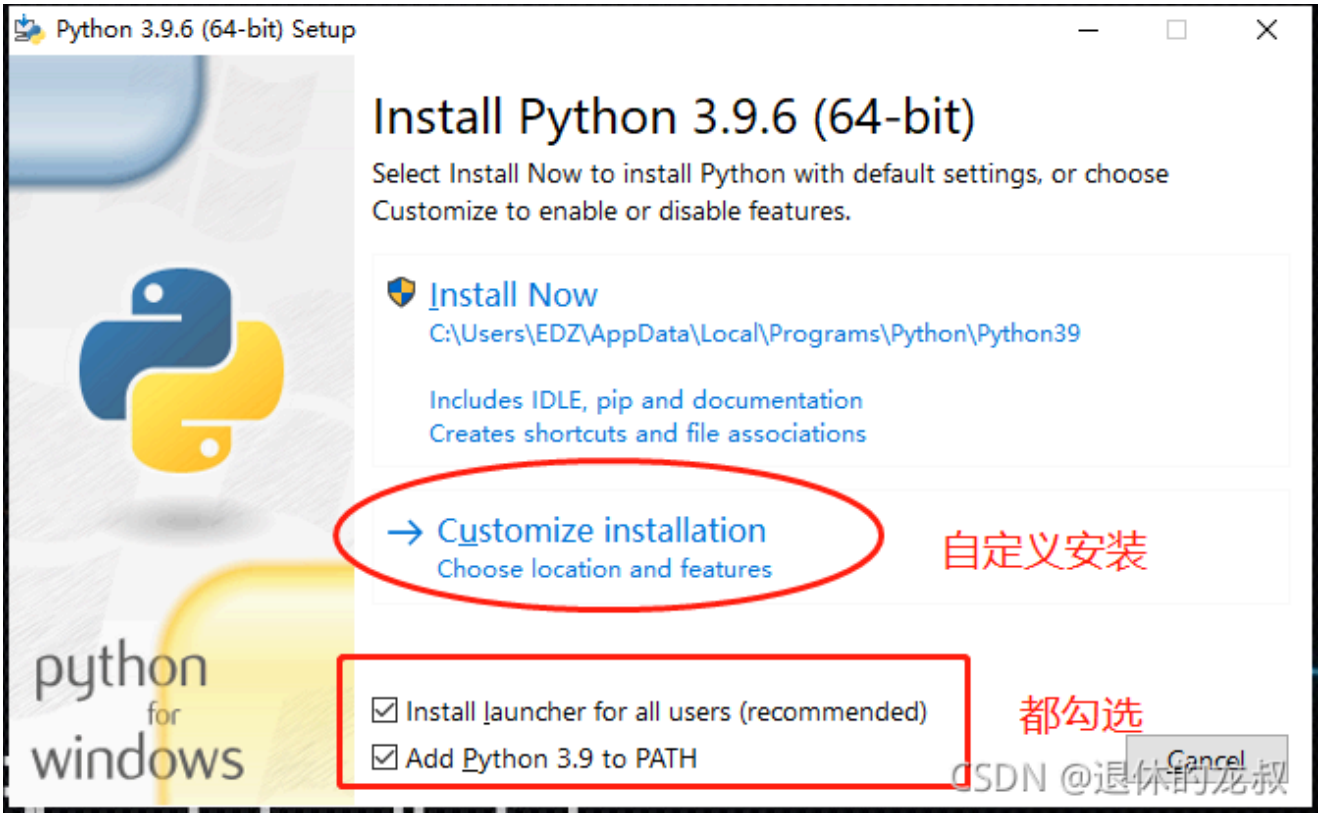

Python
PSF
Docs
PyPI
Jobs
Community
python
Donate
Search
GO
Socialize
About
Downloads
Documentation
Community
Success Stories
News
Events
Download the latest version for Windows
Download Python 3.9.7
Looking for Python with a different OS? Python for Windows, Linux/UNIX, macOS, Other
Want to help test development versions of Python? Prereleases, Docker images
Looking for Python 2.7? See below for specific releases
Active Python Releases
For more information visit the Python Developer's Guide.
Python version | Maintenance status | First released | End of support | Release schedule
3.9 | bugfix | 2020-10-05 | 2025-10 | PEP 596
3.8 | security | 2019-10-14 | 2024-10 | PEP 569
3.7 | security | 2018-06-27 | 2023-06-27 | PEP 537
3.6 | security | 2016-12-23 | 2021-12-23 | PEP 494
2.7 | end-of-life | 2010-07-03 | 2020-01-01 | PEP 373
CSDN @退休的龙叔

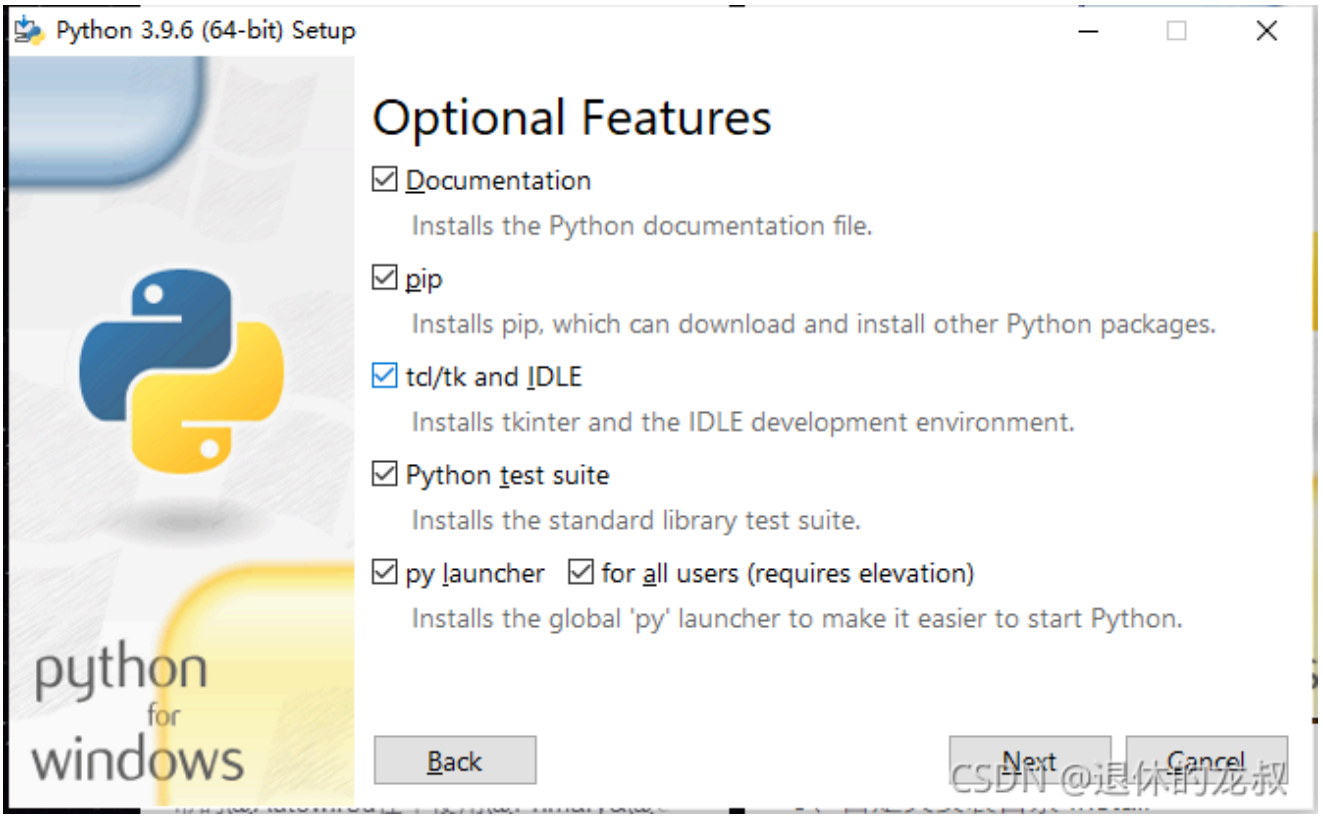

Download Windows installer (64-bit)
Python 3.8.12 - Aug. 30, 2021
Note that Python 3.8.12 cannot be used on Windows XP or earlier.
No files for this release.
Python 3.9.6 - June 28, 2021
Note that Python 3.9.6 cannot be used on Windows 7 or earlier.
Download Windows embeddable package (32-bit)
Download Windows embeddable package (64-bit)
Download Windows help file
Download Windows installer (32-bit)
Download Windows installer (64-bit)
点击这里进行下载
Python 3.8.11 - June 28, 2021
Note that Python 3.8.11 cannot be used on Windows XP or earlier.
CSDN @退休的龙叔

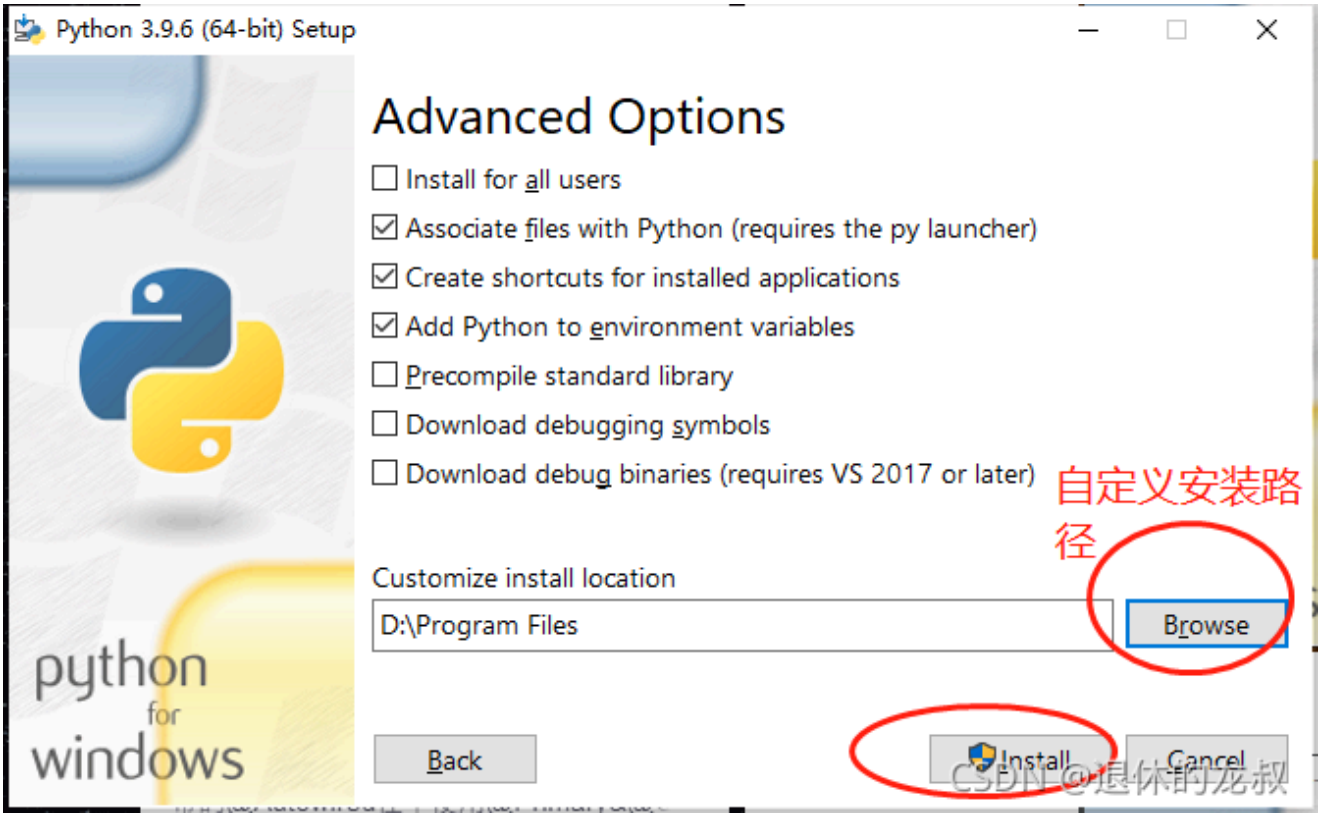


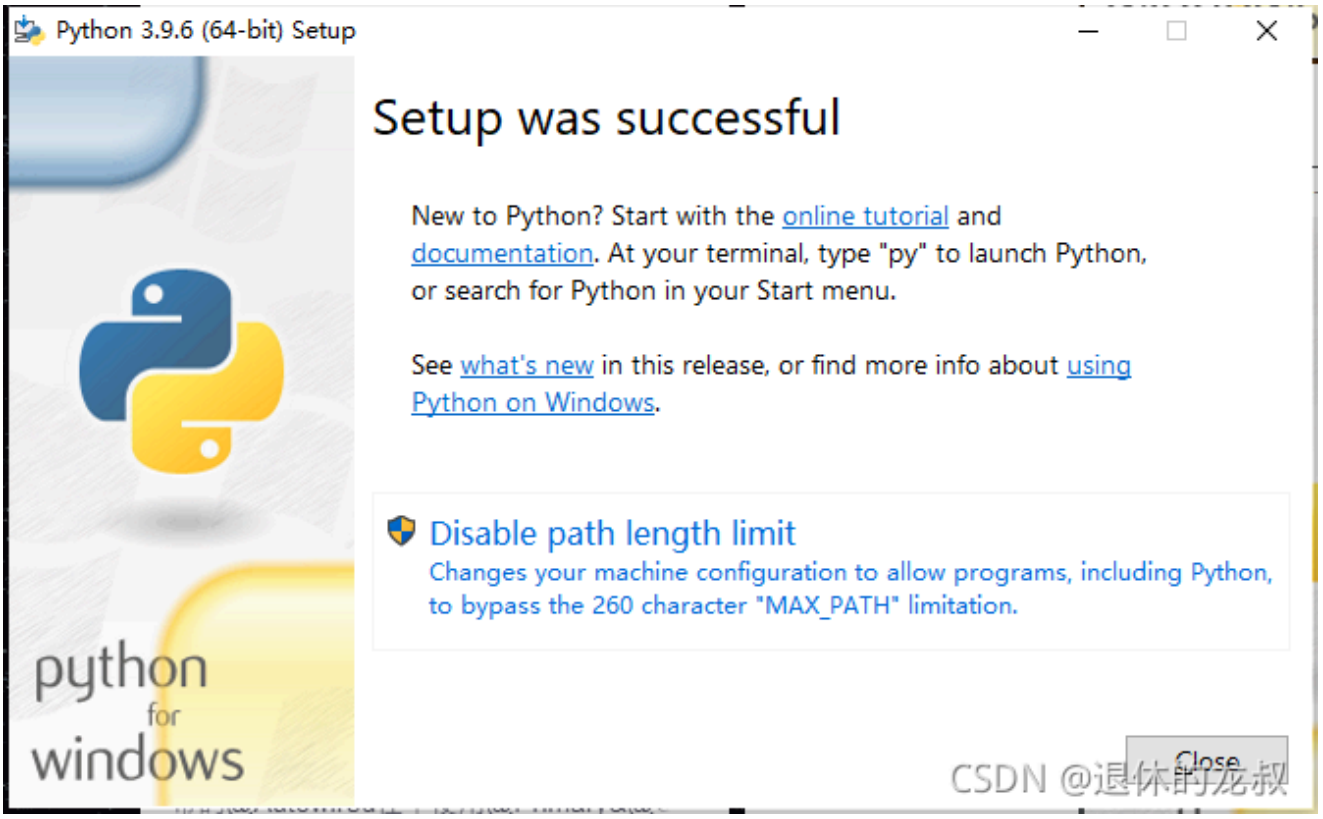
点击它

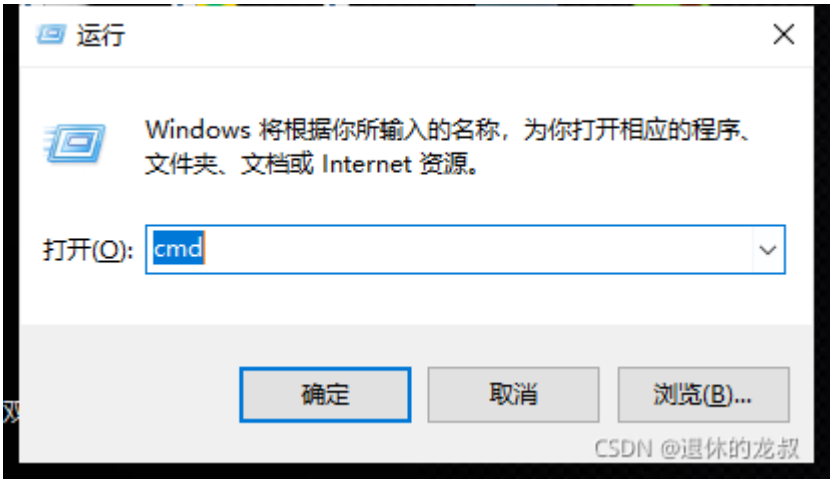
CSDN @退休的龙叔

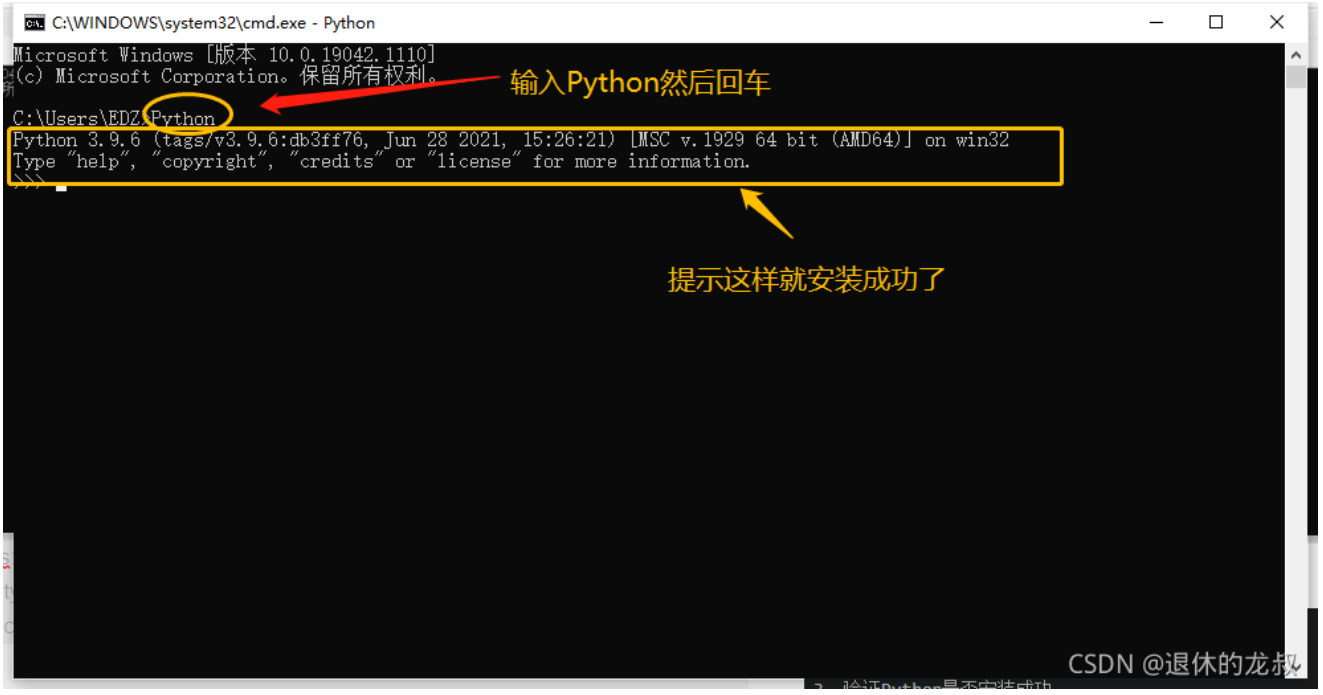


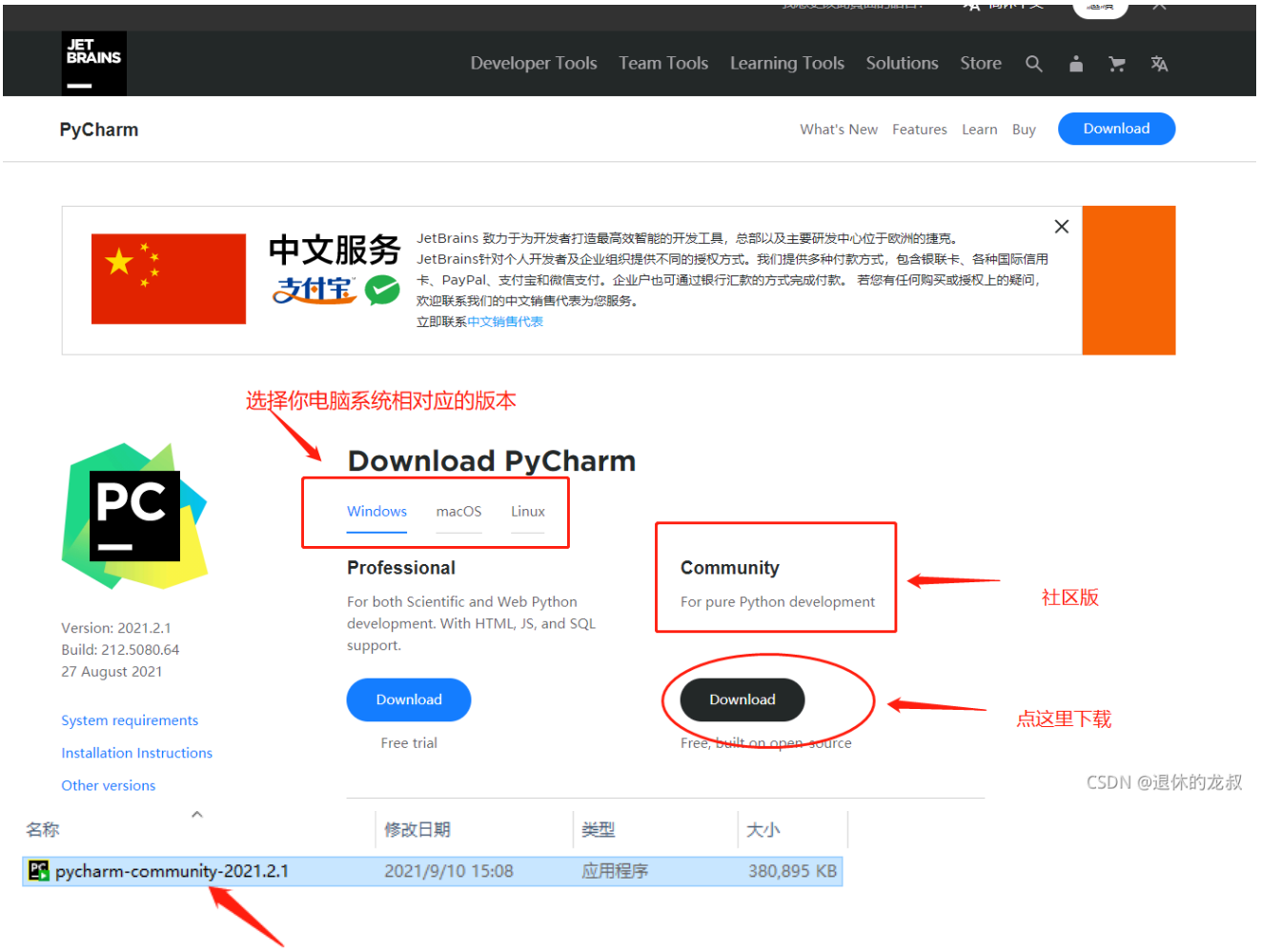

Python 3.9.6 (64-bit) Setup
Optional Features
Documentation
Installs the Python documentation file.
pip
Installs pip, which can download and install other Python packages.
tcl/tk and IDLE
Installs tkinter and the IDLE development environment.
Python test suite
Installs the standard library test suite.
py launcher
for all users (requires elevation)
Installs the global 'py' launcher to make it easier to start Python.
python
for
windows
Back
Next
Cancel
CSDN @退休的龙叔


Python 3.9.6 (64-bit) Setup
Advanced Options
Install for all users
Associate files with Python (requires the py launcher)
Create shortcuts for installed applications
Add Python to environment variables
Precompile standard library
Download debugging symbols
Download debug binaries (requires VS 2017 or later)
自定义安装路径
Customize install location
D:\Program Files
Browse
python
for
windows
Back
Install
Cancel
CSDN @退休的龙叔

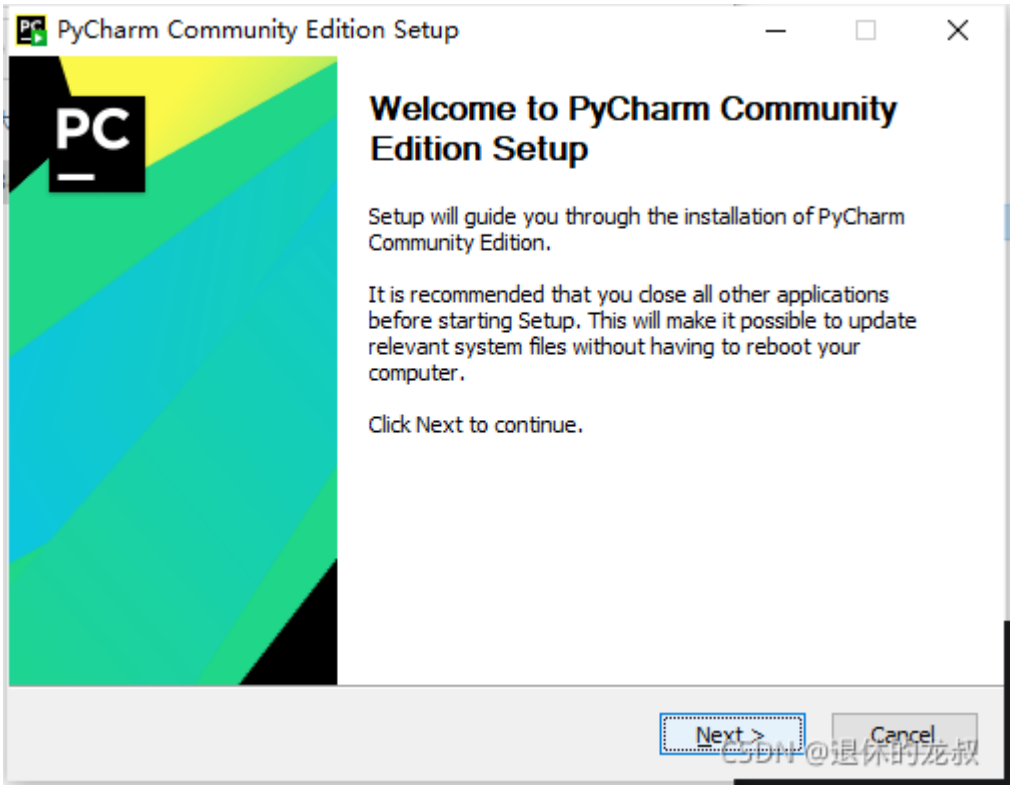

Python 3.9.6 (64-bit) Setup
Setup was successful
New to Python? Start with the online tutorial and documentation. At your terminal, type "py" to launch Python, or search for Python in your Start menu.
See what's new in this release, or find more info about using Python on Windows.
Disable path length limit
Changes your machine configuration to allow programs, including Python, to bypass the 260 character "MAX_PATH" limitation.
python for windows
Close
CSDN @退休的龙叔

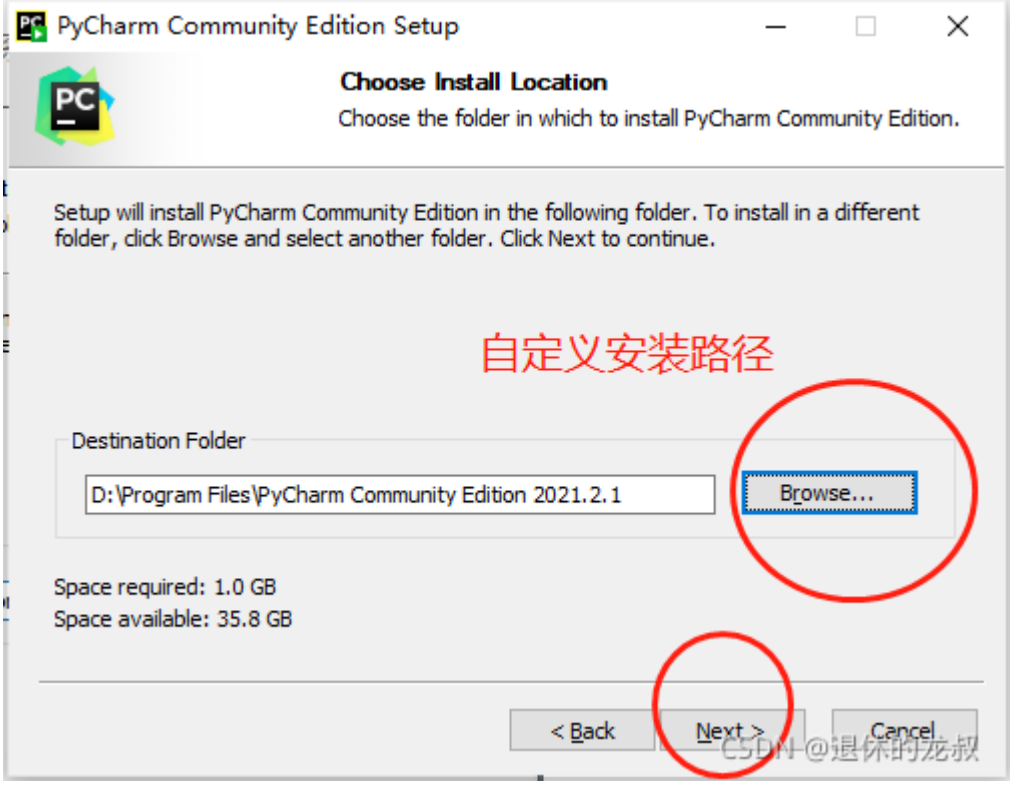

运行
Windows 将根据你所输入的名称，为你打开相应的程序、文件夹、文档或 Internet 资源。
打开(O):
cmd
确定
取消
浏览(B)...
CSDN @退休的龙叔

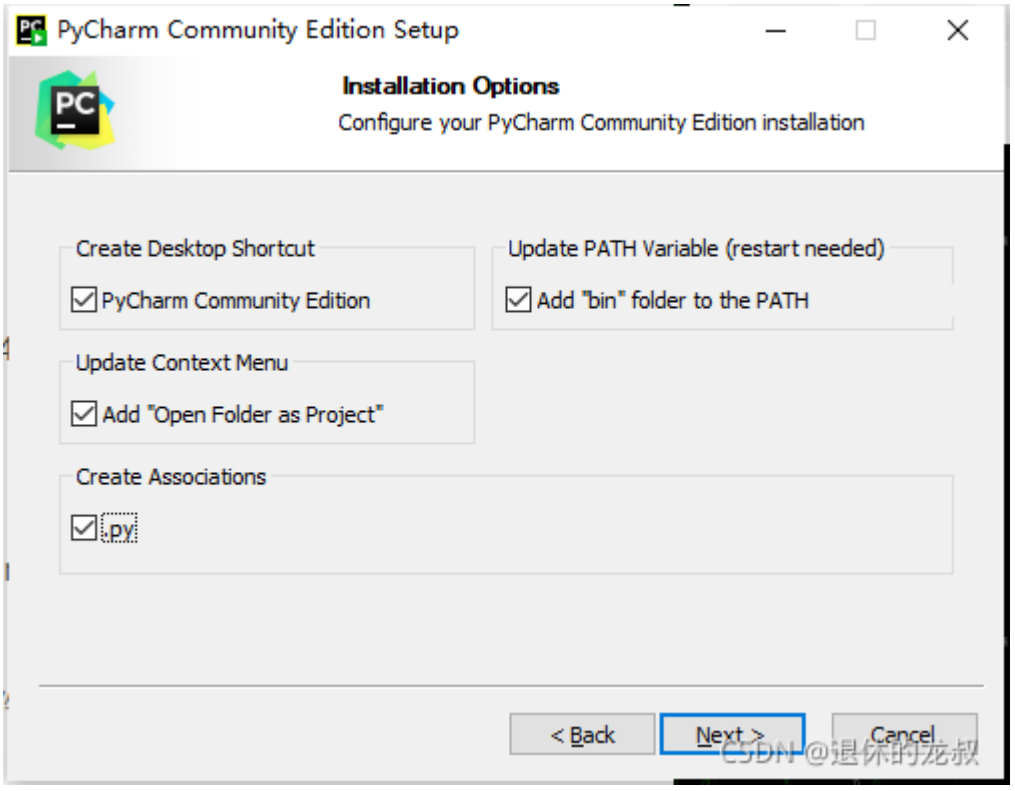

C:\WINDOWS\system32\cmd.exe - Python
Microsoft Windows [版本 10.0.19042.1110]
(c) Microsoft Corporation。保留所有权利。
C:\Users\EDZ>Python
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
输入Python然后回车
提示这样就安装成功了
CSDN @退休的龙叔

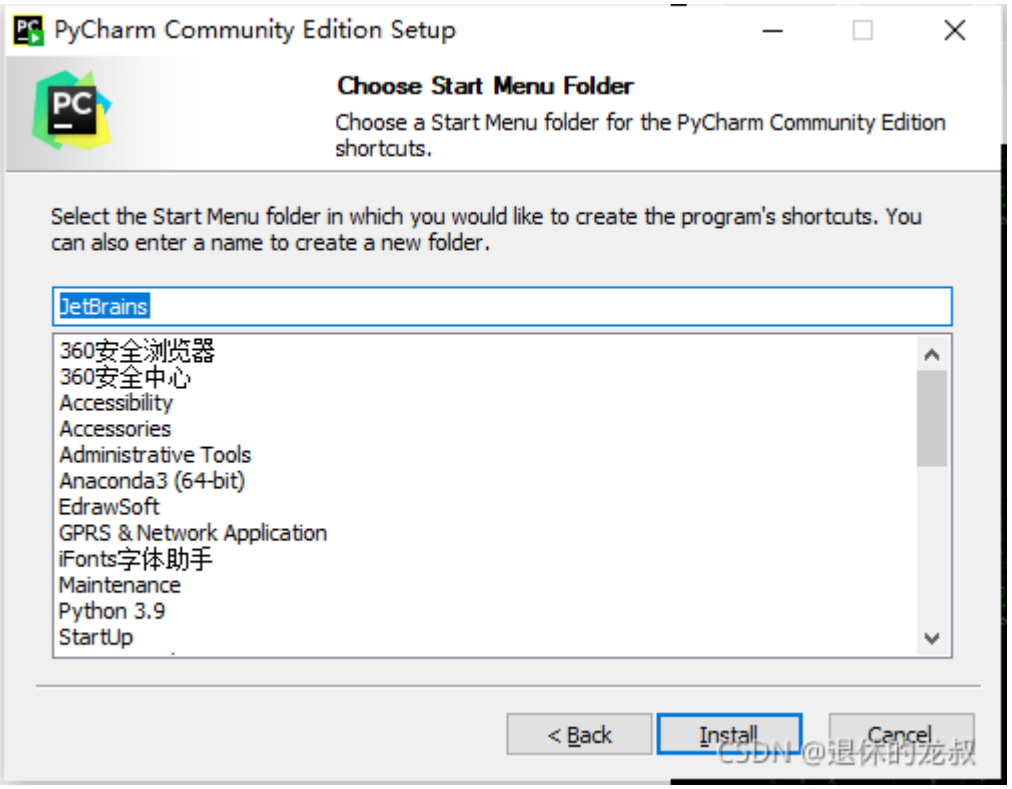

JET BRAINS
Developer Tools
Team Tools
Learning Tools
Solutions
Store
PyCharm
What's New
Features
Learn
Buy
Download
中文服务
支付宝
JetBrains 致力于为开发者打造最高效智能的开发工具，总部以及主要研发中心位于欧洲的捷克。
JetBrains针对个人开发者及企业组织提供不同的授权方式。我们提供多种付款方式，包含银联卡、各种国际信用卡、PayPal、支付宝和微信支付。企业户也可通过银行汇款的方式完成付款。 若您有任何购买或授权上的疑问，欢迎联系我们的中文销售代表为您服务。
立即联系中文销售代表
选择你电脑系统相对应的版本
Download PyCharm
Windows
macOS
Linux
Professional
For both Scientific and Web Python development. With HTML, JS, and SQL support.
Download
Free trial
Community
For pure Python development
社区版
Download
点这里下载
Free, built on open-source
Version: 2021.2.1
Build: 212.5080.64
27 August 2021
System requirements
Installation Instructions
Other versions
CSDN @退休的龙叔
名称
修改日期
类型
大小
pycharm-community-2021.2.1
2021/9/10 15:08
应用程序
380,895 KB

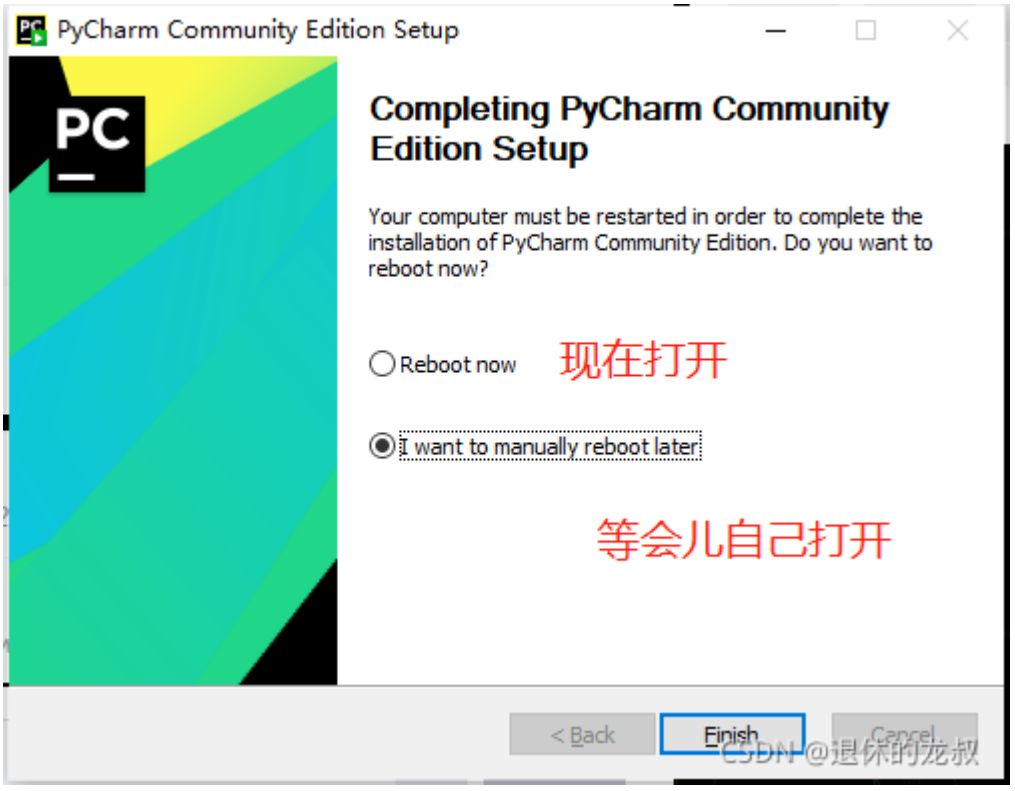
CSDN @退休的龙叔

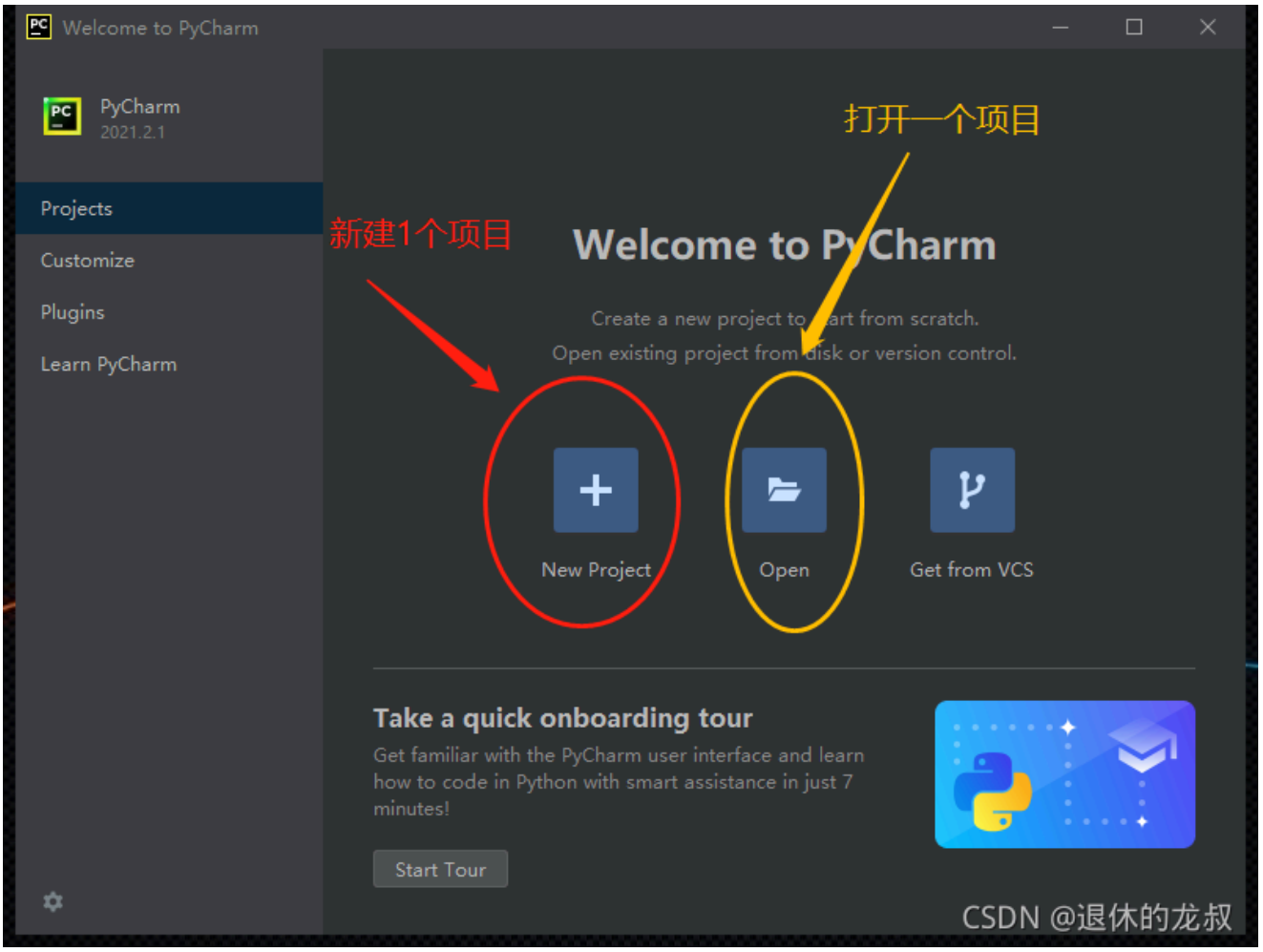

PyCharm Community Edition Setup
Welcome to PyCharm Community Edition Setup
Setup will guide you through the installation of PyCharm Community Edition.
It is recommended that you close all other applications before starting Setup. This will make it possible to update relevant system files without having to reboot your computer.
Click Next to continue.
Next >
Cancel
CSDN @退休的龙叔

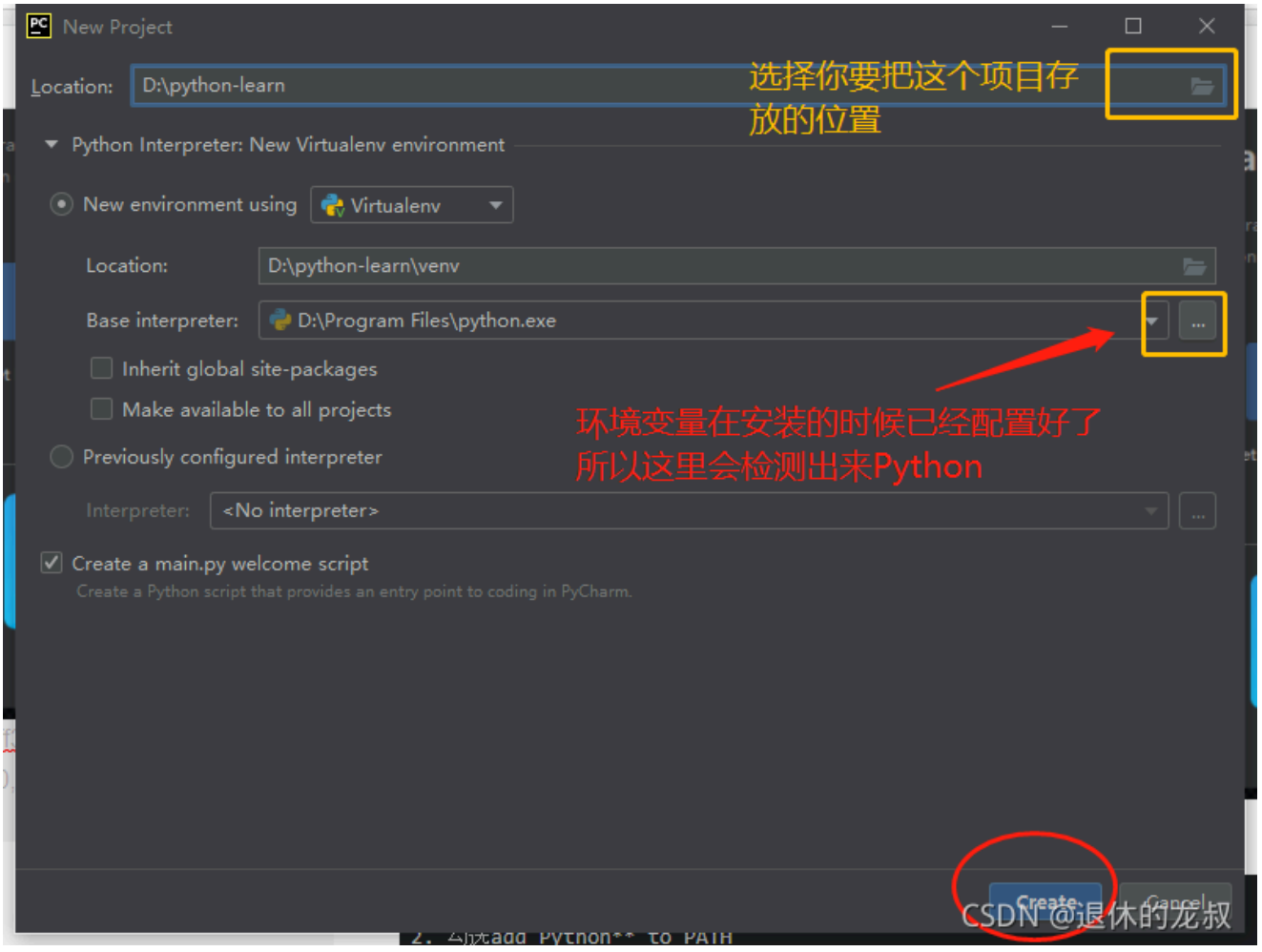

PyCharm Community Edition Setup
Choose Install Location
Choose the folder in which to install PyCharm Community Edition.
Setup will install PyCharm Community Edition in the following folder. To install in a different folder, click Browse and select another folder. Click Next to continue.
自定义安装路径
Destination Folder
D:\Program Files\PyCharm Community Edition 2021.2.1
Browse...
Space required: 1.0 GB
Space available: 35.8 GB
< Back
Next >
Cancel
CSDN @退休的龙叔

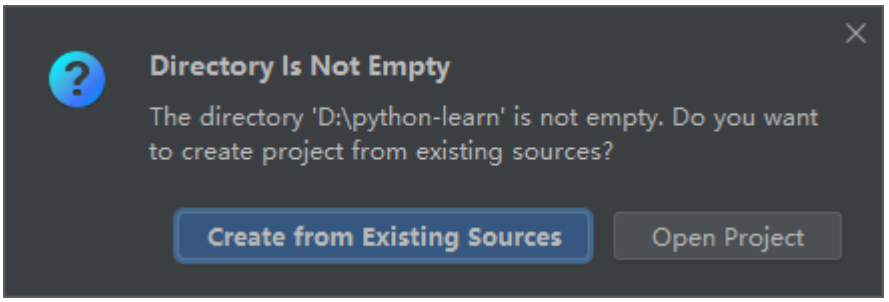

PyCharm Community Edition Setup
Installation Options
Configure your PyCharm Community Edition installation
Create Desktop Shortcut
PyCharm Community Edition
Update PATH Variable (restart needed)
Add "bin" folder to the PATH
Update Context Menu
Add "Open Folder as Project"
Create Associations
.py
< Back
Next >
Cancel
CSDN @退休的龙叔

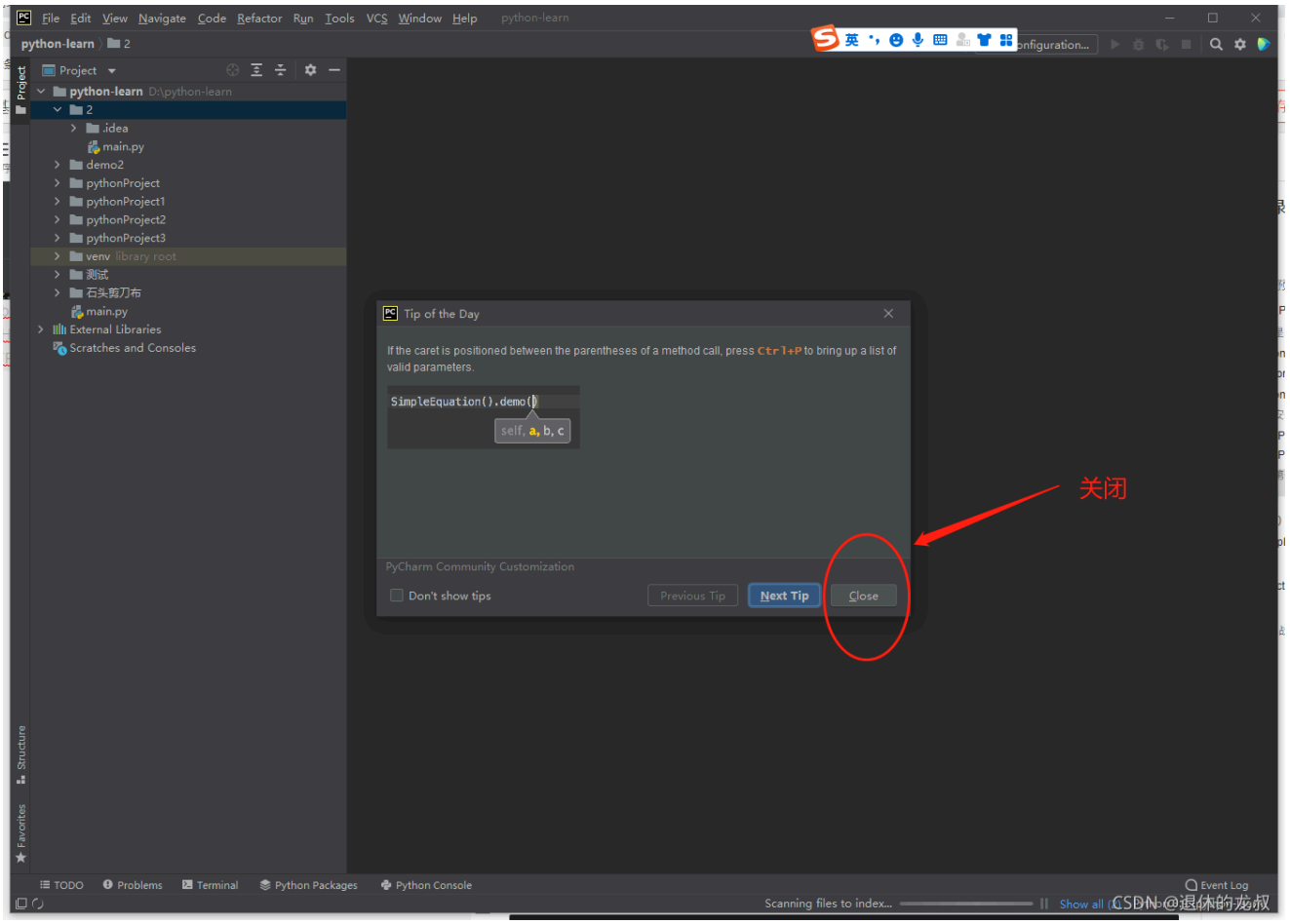

PyCharm Community Edition Setup
Choose Start Menu Folder
Choose a Start Menu folder for the PyCharm Community Edition shortcuts.
Select the Start Menu folder in which you would like to create the program's shortcuts. You can also enter a name to create a new folder.
JetBrains
360安全浏览器
360安全中心
Accessibility
Accessories
Administrative Tools
Anaconda3 (64-bit)
EdrawSoft
GPRS & Network Application
iFonts字体助手
Maintenance
Python 3.9
StartUp
< Back
Install
Cancel
CSDN @退休的龙叔

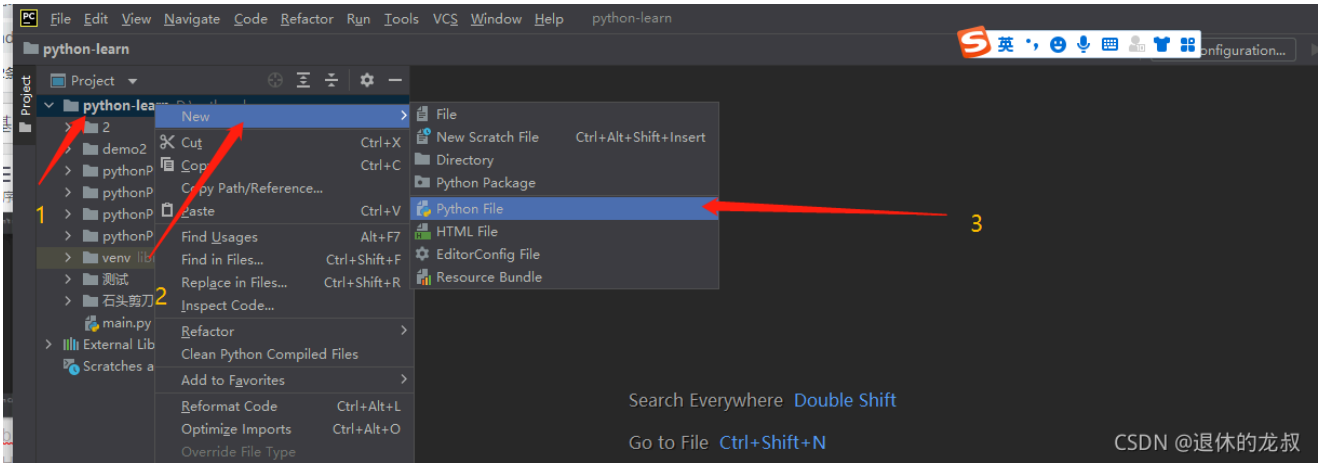

PyCharm Community Edition Setup
Completing PyCharm Community Edition Setup
Your computer must be restarted in order to complete the installation of PyCharm Community Edition. Do you want to reboot now?
Reboot now
现在打开
I want to manually reboot later
等会儿自己打开
< Back
Finish
Cancel
CSDN @退休的龙叔

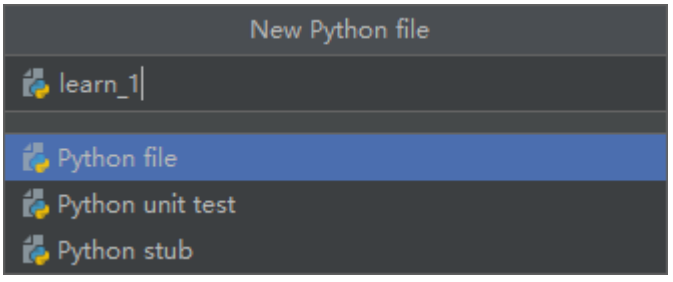

Welcome to PyCharm
PyCharm
2021.2.1
Projects
Customize
Plugins
Learn PyCharm
新建1个项目
打开一个项目
Welcome to PyCharm
Create a new project to start from scratch.
Open existing project from disk or version control.
New Project
Open
Get from VCS
Take a quick onboarding tour
Get familiar with the PyCharm user interface and learn how to code in Python with smart assistance in just 7 minutes!
Start Tour
CSDN @退休的龙叔

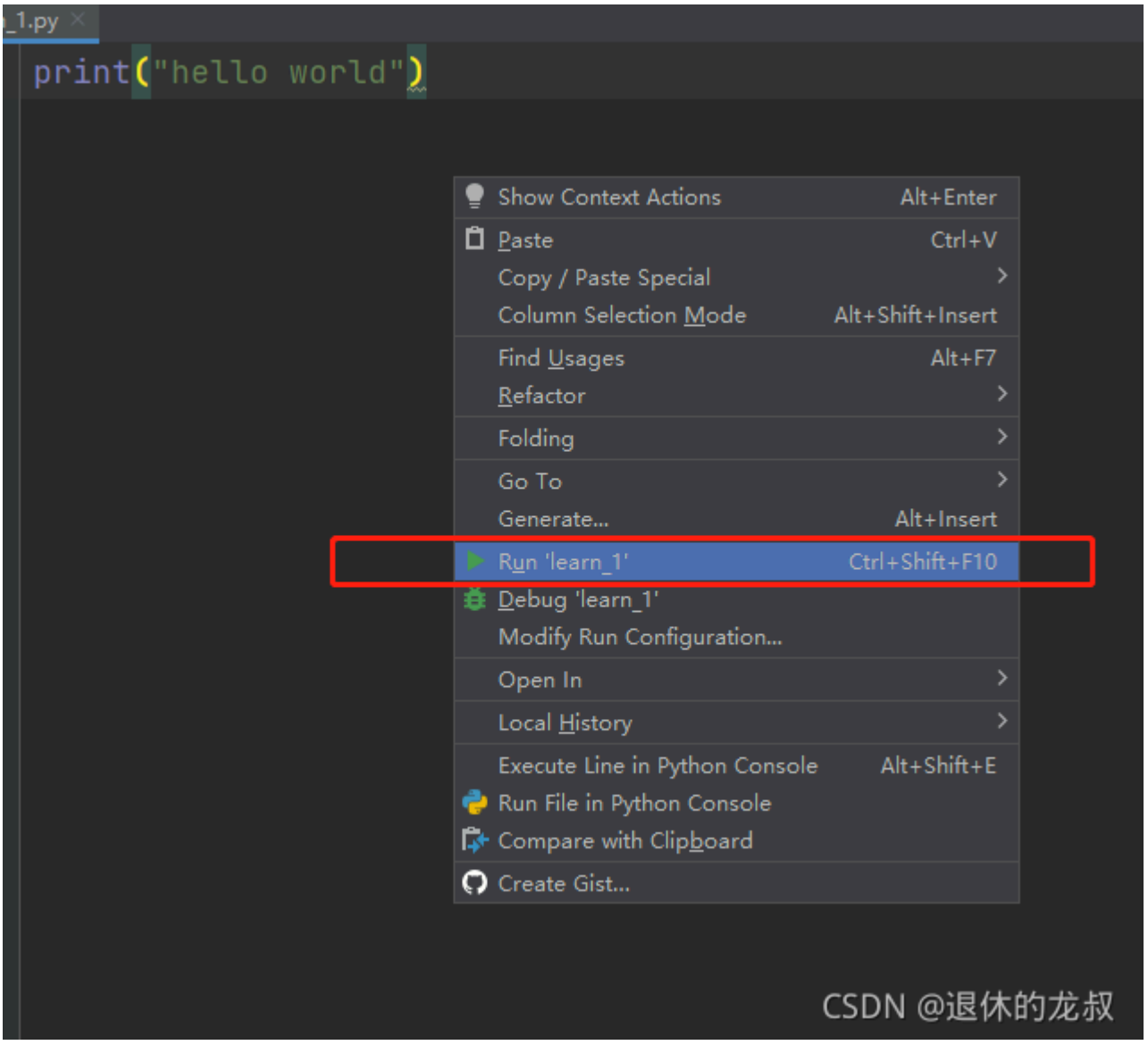

New Project
Location: D:\python-learn
选择你要把这个项目存放的位置
Python Interpreter: New Virtualenv environment
New environment using Virtualenv
Location: D:\python-learn\venv
Base interpreter: D:\Program Files\python.exe
Inherit global site-packages
Make available to all projects
Previously configured interpreter
环境变量在安装的时候已经配置好了
所以这里会检测出来Python
Interpreter: <No interpreter>
Create a main.py welcome script
Create a Python script that provides an entry point to coding in PyCharm.
Create
Cancel
CSDN @退休的龙叔

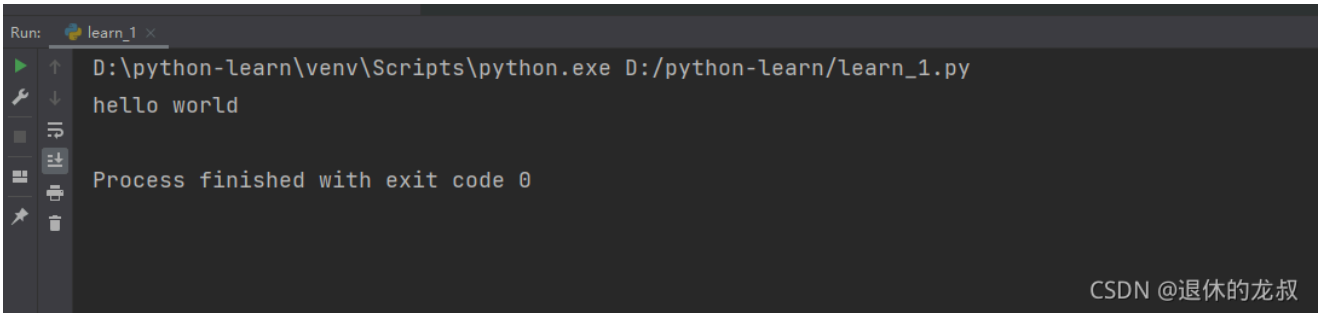

Directory Is Not Empty
The directory 'D:\python-learn' is not empty. Do you want to create project from existing sources?
Create from Existing Sources
Open Project


Tip of the Day
If the caret is positioned between the parentheses of a method call, press Ctrl+P to bring up a list of valid parameters.
SimpleEquation().demo()
self, a, b, c
PyCharm Community Customization
Don't show tips
Previous Tip
Next Tip
Close
关闭
CSDN @退休的龙叔


New
Cut Ctrl+X
Copy Ctrl+C
Copy Path/Reference...
Paste Ctrl+V
Find Usages Alt+F7
Find in Files... Ctrl+Shift+F
Replace in Files... Ctrl+Shift+R
Inspect Code...
Refactor
Clean Python Compiled Files
Add to Favorites
Reformat Code Ctrl+Alt+L
Optimize Imports Ctrl+Alt+O
Override File Type
File
New Scratch File Ctrl+Alt+Shift+Insert
Directory
Python Package
Python File
HTML File
EditorConfig File
Resource Bundle
1
2
3
Search Everywhere Double Shift
Go to File Ctrl+Shift+N
CSDN @退休的龙叔


New Python file
learn_1
Python file
Python unit test
Python stub





序号	函数	说明
1	append()	向列表中添加对象，并添加到末尾
2	extend(可迭代对象)	将可迭代对象中数据分别添加到列表中，并添加到末尾
3	insert(下标,对象)	向指定下标位置添加对象
4	clear()	清空列表
5	pop()	删除下标指定的元素，如果不加下标则删除最后一个元素
6	remove(对象)	删除指定的对象
7	del	删除变量或指定下表的值

8	copy()	浅拷贝
9	count(对象)	返回对象在列表中出现的次数
10	index(value,开始下标,结束下标)	元素出现的第一次下标位置，也可自定义范围
11	reverse()	原地翻转
12	sort (key=None, reverse=False)	快速排序，默认从小到大排序，key:算法
13	len()	获取列表的长度（元素）

@退休的龙叔

3. 列表

序号	函数	说明
1	append()	向列表中添加对象，并添加到末尾
2	extend(可迭代对象)	将可迭代对象中数据分别添加到列表中，并添加到末尾
3	insert(下标,对象)	向指定下标位置添加对象
4	clear()	清空列表
5	pop()	删除下标指定的元素，如果不加下标则删除最后一个元素
6	remove(对象)	删除指定的对象
7	del	删除变量或指定下表的值
8	copy()	浅拷贝
9	count(对象)	返回对象在列表中出现的次数
10	index(value,开始下标,结束下标)	元素出现的第一次下标位置，也可自定义范围
11	reverse()	原地翻转
12	sort (key=None, reverse=False)	快速排序，默认从小到大排序，key:算法
13	len()	获取列表的长度（元素）

@退休的龙叔

序号	函数	说明
1	capitalize()	把字符串的第一个字符改为大写，后面的小写
2	casefold()	把整个字符串都小写
3	encode()	编码 str--bytes（二进制字符串）
4	decode()	解码
5	count(sub,start,stop)	返回字符（sub）出现的次数，star：开始下标，stop:结束下标
6	find(sub,start,stop)	返回sub第一次出现的下标,查不到返回-1
7	index(sub,start,stop)	返回sub第一次出现的下标
8	upper()	将字符串转为大写
9	lower()	将字符串转为小写

CSDN @退休的龙叔

6. 字典

序号	函数	说明
1	clear()	清空字典
2	copy()	浅拷贝
3	fromkeys(可迭代对象，value=None)	根据可迭代对象中的元素去创建字典
4	get(key,[d])	获取键所对应的值，key是键，d是提示信息
5	items()	将字典中的键值对封装成元组并放到类集合中
6	pop(key,[d])	根据键删除字典中的键值对，*key*是键，*d*是提示信息
7	values()	返回字典中的值（类集合对象）
8	popitem()	删除某个键值对
9	values()	返回字典中的值（类集合对象）

CSDN @退休的龙叔

```
def fun2(a, n):
    print(a * n)


# 打印古诗
print("白日依山尽")
fun2("+", 5)
print("黄河入海流")
fun2("-", 10)
print("欲穷千里目")
fun2("*", 15)
print("更上一层楼")
fun2("/", 20)
```

a和n是形参

+和5是实参

CSDN @退休的龙叔

```python
def fun4(a, b, c, d):
    print(a, b, c)
fun4(10, 20, 30)
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
Traceback (most recent call last):
  File "D:\python-learn\learn_1.py", line 3, in <module>
    fun4(10, 20, 30)
TypeError: fun4() missing 1 required positional argument: 'd'
```

少传了1个值给形参d，只传了3个值

```
Process finished with exit code 1
```

CSDN @退休的龙叔

```python
def fun4(a, b, c):
    print(a, b, c)

fun4(b=10, a=20, c=30)
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
20 10 30

Process finished with exit code 0
```

CSDN @退休的龙叔

```python
def fun4(a, b, c):
    print(a, b, c)

fun4(a=10, 20, c=30)
```

关键字参数不能在前面
只要把 a=10 换成 10 ,则变为位置参数在前，结果是没有报错的

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
  File "D:\python-learn\learn_1.py", line 4
    fun4(a=10, 20, c=30)
                       ^
SyntaxError: positional argument follows keyword argument

Process finished with exit code 1
```

CSDN @退休的龙叔

```python
def fun4(a=10, b=20, c=30):
    print(a, b, c)

fun4()
fun4(20, 30, 40)
fun4(b=20, c=30, a=40)
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
10 20 30
20 30 40
40 20 30
```

CSDN @退休的龙叔

```python
def fun5(*a):
    print(a)  # 元组

fun5(10, 20, 30)
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
(10, 20, 30)

Process finished with exit code 0
```

CSDN @退休的龙叔

```python
def fun6(**a):
    print(a)

fun6(b=10, c=20, d=30)
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
{'b': 10, 'c': 20, 'd': 30}

Process finished with exit code 0
```

CSDN @退休的龙叔

```python
def fun6(*a, b):
    print(a, b)

fun6(10, 20, b=30)

def fun6(b, *a):
        print(a, b)

fun6(10, 20, 30)
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
(10, 20) 30
(20, 30) 10

Process finished with exit code 0
```

CSDN @退休的龙叔

```python
def fun6(b, *a):
    """
    fun6是个函数
    a:可变成参数
    b:位置参数
    """
    print(a, b)

fun6(10, 20, 30)
```

CSDN @退休的龙叔

Project

python-learn D:\python-learn
- 2
- demo2
- pythonProject
- pythonProject1
- pythonProject2
- pythonProject3
- venv library root
- 测试
- 石头剪刀布
- learn_1.py
- main.py

External Libraries

Scratches and Consoles

learn_1.py | index.html

```python
def fun6(b, *a):
    """
    fun6是个函数
    a:可变成参数
    b:位置参数
    """
    print(a, b)

fun6(10, 20, 30)
print(fun6.__doc__)
```

learn_1

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
(20, 30) 10

    fun6是个函数
    a:可变成参数
    b:位置参数
```

CSDN @退休的龙叔

```python
def fun6(b, *a):
    """
    fun6是个函数
    a:可变成参数
    b:位置参数
    """
    print(a, b)

help(fun6)
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
Help on function fun6 in module __main__:

fun6(b, *a)
    fun6是个函数
    a:可变成参数
    b:位置参数


Process finished with exit code 0
```

CSDN @退休的龙叔

```python
def fun1():
    global b
    b = 100
    print(b)

fun1()
print(b)
```

局部变量b被global声明后
在函数外部也能使用

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
100
100

Process finished with exit code 0
```

CSDN @退休的龙叔

```python
def fun1():
    b = 100
    def fun2():
        nonlocal b
        b = 300
        print(b)
    fun2()
    print(b)
fun1()
print(b)
```

声明之后 b在函数内变成了300
如果不声明，则在这里输出的是100

b还是局部变量

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
Traceback (most recent call last):
  File "D:\python-learn\learn_1.py", line 10, in <module>
    print(b)
NameError: name 'b' is not defined
300
300

Process finished with exit code 1
```

CSDN @退休的龙叔

```python
def fun1():
    b = 100
    def fun2():
        nonlocal b
        b = b + 1
        print(b)
    fun2()
    print(b)

fun1()
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
101
101

Process finished with exit code 0
```

CSDN @退休的龙叔

```python
def fun1():
    a=100
    def fun2():
        print(a)
        return "哈哈"
    return fun2

print(fun1()())
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
100
哈哈

Process finished with exit code 0
```

CSDN @退休的龙叔

```python
def fun1(b):
    a = 100
    print(b)

    def fun2(c):
        print(c)
        return "哈哈"

    return fun2


print(fun1(200)(300))
```

```
D:\python-learn\venv\Scripts\python.exe D:/python-learn/learn_1.py
200
300
哈哈

Process finished with exit code 0
```

CSDN @退休的龙叔

虽然头发越来越少了……

但只要还有人点赞、关注和收藏

我就会继续更新出很哇塞的内容!